Explore more resources
Altera® Design Hub

External Memory Interfaces Agilex™ 7 M-Series FPGA IP Design Example User Guide

Updated for Quartus® Prime Design Suite: **25.3**

IP Version: **4.1.0**

Online Version
Send Feedback

772632
2025.09.29

Contents

1. About the External Memory Interfaces Agilex™ 7 M-Series FPGA IP.....3
1.1. Release Information.....3
2. Design Example Quick Start Guide for External Memory Interfaces Agilex™ 7 M-Series FPGA IP 5
2.1. Creating an EMIF Project.....5
2.2. Generating and Configuring the EMIF IP.....6
2.2.1. Agilex 7 M-Series EMIF Parameter Editor Guidelines.....8
2.2.2. Using Custom Memory Timing and I/O Configuration.....10
2.3. Parameterizing the External Memory Interface for HPS IP.....13
2.3.1. Parameterization Flow.....14
2.3.2. Parameterizing the EMIF HPS IP.....15
2.3.3. Parameterizing the EMIF IP.....15
2.4. Configuring DQ Pin Swizzling.....16
2.4.1. Example: DQ Pin Swizzling Within DQS group for x32 DDR4 interface.....17
2.4.2. Example: Byte Swizzling for a x32 DDR4 interface, using a memory device of x8 width.....20
2.4.3. Combining Pin and Byte Swizzling.....21
2.4.4. Example: Swizzling for a x32 + ECC interface.....23
2.4.5. Example: Swizzling for a 2Ch x32 + ECC interface.....24
2.4.6. Example: Byte Swizzling for Lockstep Configuration.....26
2.5. Generating the Synthesizable EMIF Design Example.....31
2.6. Back Annotating Pin Placement and I/O Standard Assignments.....34
2.7. Generating the EMIF Design Example for Simulation.....37
2.8. Pin Placement for Agilex 7 M-Series EMIF IP.....39
2.9. Compiling the Agilex 7 M-Series EMIF Design Example.....40
2.10. Using the EMIF Design Example with the Test Engine IP.....41
2.11. Generating the EMIF Design Example with the Performance Monitor.....42
3. Design Example Description for External Memory Interfaces Agilex 7 M-Series FPGA IP 45
3.1. Synthesis Design Example.....45
3.2. Simulation Design Example.....48
3.2.1. Running Simulation.....50
3.3. Example Design Tab.....52
4. Document Revision History for External Memory Interfaces Agilex 7 M-Series FPGA IP Design Example User Guide.....53

1. About the External Memory Interfaces Agilex™ 7 M-Series FPGA IP

1.1. Release Information

IP versions are the same as the Quartus® Prime Design Suite software versions up to v19.1. From Quartus Prime Design Suite software version 19.2 or later, IP cores have a new IP versioning scheme.

The IP versioning scheme (X.Y.Z) number changes from one software version to another. A change in:

- X indicates a major revision of the IP. If you update your Altera Quartus Prime software, you must regenerate the IP.
- Y indicates the IP includes new features. Regenerate your IP to include these new features.
- Z indicates the IP includes minor changes. Regenerate your IP to include these changes.

IP Name	IP Version	Quartus Version	Release Date
External Memory Interfaces (EMIF) IP - DDR4 Component	4.1.0	25.3	2025.09.29
External Memory Interfaces (EMIF) IP - DDR4 DIMM	4.1.0	25.3	2025.09.29
External Memory Interfaces (EMIF) IP - DDR5 Component	4.1.0	25.3	2025.09.29
External Memory Interfaces (EMIF) IP - DDR5 DIMM	4.1.0	25.3	2025.09.29
External Memory Interfaces (EMIF) IP - LPDDR5 Component	4.1.0	25.3	2025.09.29

Note: This documentation is preliminary and subject to change.

Related Information

- External Memory Interfaces Agilex™ 7 M-Series FPGA IP User Guide
  Provides architecture, parameterization, and usage information for the Agilex™ 7 FPGA EMIF IP for the DDR4, DDR5, and LPDDR5 memory protocols.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

- External Memory Interfaces Agilex™ 7 M-Series FPGA IP Release Notes
  Release notes for the Agilex™ 7 M-Series EMIF IP.

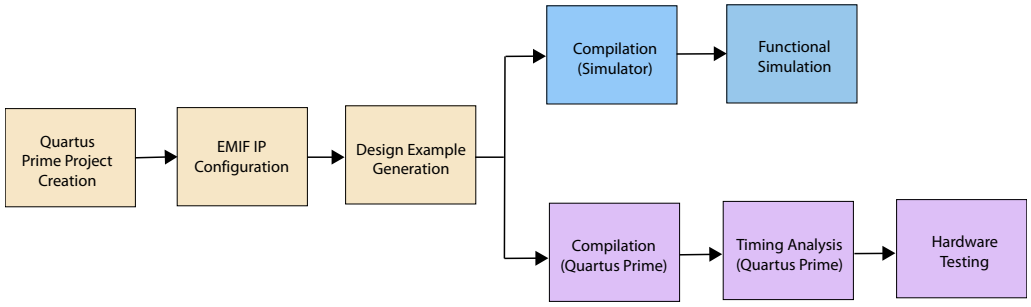
2. Design Example Quick Start Guide for External Memory Interfaces Agilex™ 7 M-Series FPGA IP

An automated design example flow is available for Agilex™ 7 M-Series external memory interfaces.

The **Example Designs** tab in the EMIF IP parameter editor allows you to set a variety of parameters to generate the synthesis and simulation design example file sets which you can use to validate your EMIF IP.

You can generate a design example that matches the Altera FPGA development kit, or for any EMIF IP that you generate. You can use the design example to assist your evaluation, or as a starting point for your own system.

Figure 1. General Design Example Workflows



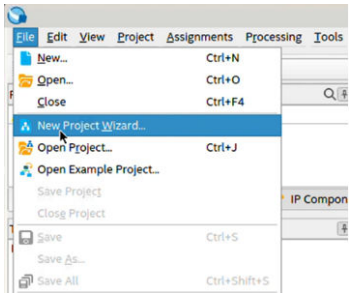
2.1. Creating an EMIF Project

You must create a Quartus Prime project before generating the Agilex 7 M-Series EMIF IP design example.

1. Launch the Quartus Prime software and select **File ➢ New Project Wizard**. Click **Next**.

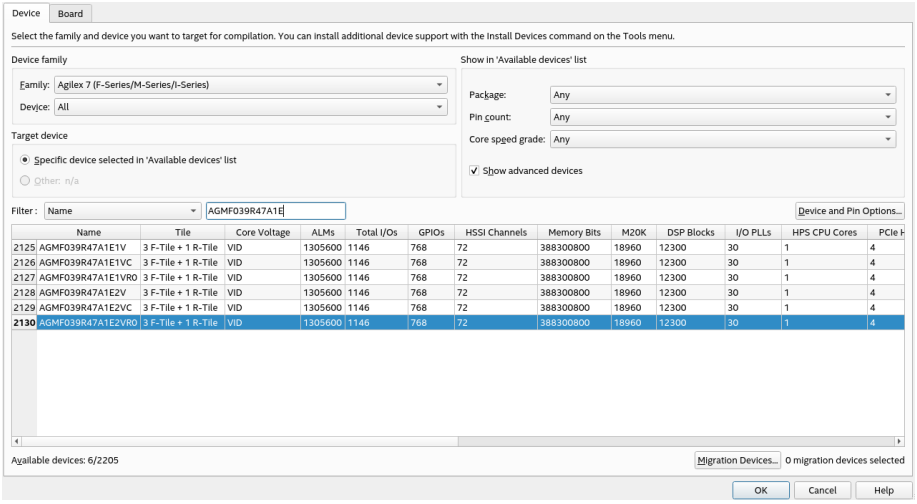
© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Figure 2. File Menu



2. Specify a directory (*<user project directory>*), a name for the Quartus Prime project (*<user project name>*), and a top-level design entity name (*<user top-level instance name>*) that you want to create.
3. Verify that **Empty Project** is selected.
4. Under **Family**, select **Agilex 7**.
5. Under **Name filter**, type the device part number.
6. Under **Available devices**, select the appropriate device.

Figure 3. Family, Device, and Board Settings



7. Click **Finish**.

2.2. Generating and Configuring the EMIF IP

The Agilex 7 M-Series External Memory Interfaces (EMIF) IP consists of 5 IPs, according to technology.

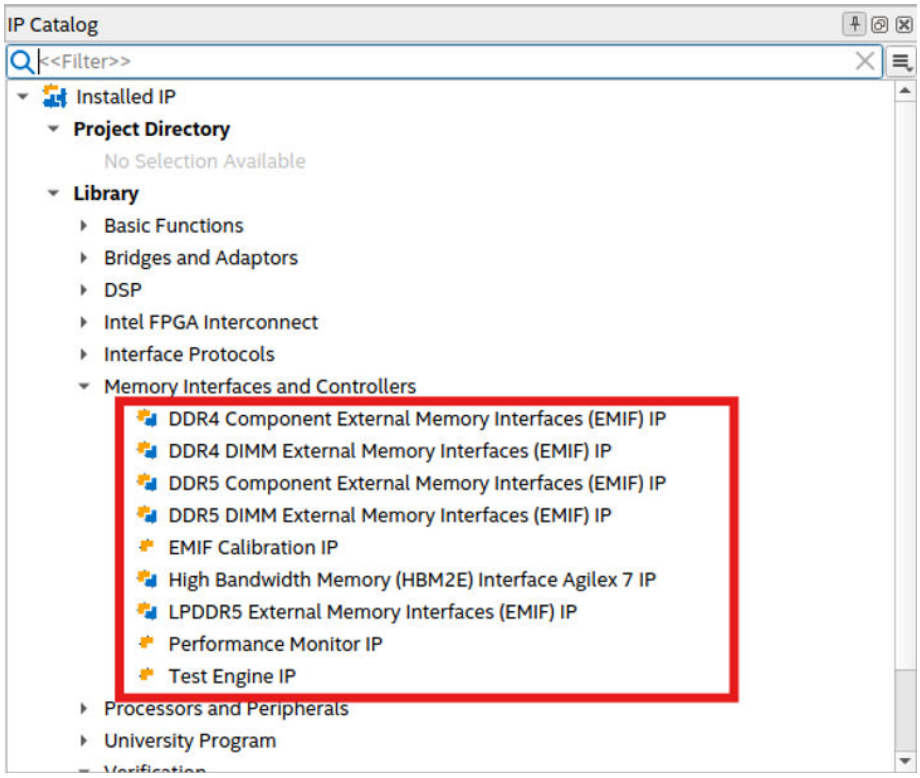
The available IPs are:

- DDR4 Component External Memory Interfaces (EMIF) IP
- DDR4 DIMM External Memory Interfaces (EMIF) IP
- DDR5 Component External Memory Interfaces (EMIF) IP
- DDR5 DIMM External Memory Interfaces (EMIF) IP
- LPDDR5 External Memory Interfaces (EMIF) IP

The following steps illustrate how to generate and configure the EMIF IP. This walkthrough creates a DDR5 – Component interface, but the steps are similar for other protocols. (These steps follow the IP Catalog (standalone) flow; if you choose to use the Platform Designer (system) flow instead, the steps are similar.)

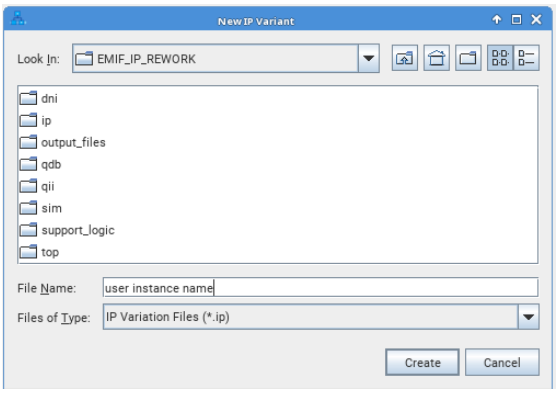
1. In the **IP Catalog window**, select **DDR5 Component External Memory Interfaces (EMIF) IP**. (If the **IP Catalog window** is not visible, select **View ➤ IP Catalog**.)

Figure 4. IP Catalog



2. In the **IP Parameter Editor**, provide an entity name for the EMIF IP (the name that you provide here becomes the file name for the IP) and specify a directory. Click **Create**.

Figure 5. Specifying a File name



3. The parameter editor has multiple tabs where you must configure parameters to reflect your EMIF implementation.

2.2.1. Agilex 7 M-Series EMIF Parameter Editor Guidelines

This topic provides high-level guidance for parameterizing the tabs in the Agilex 7 M-Series EMIF IP parameter editor.

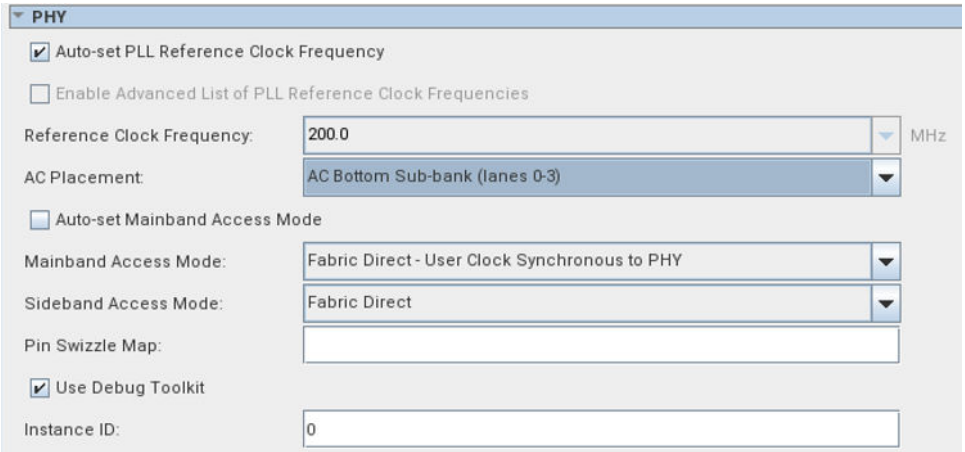
Table 1. EMIF Parameter Editor Guidelines

Section Name	Guidelines
Memory Device	Ensure that you correctly enter the following parameters: • Number of channels. (If the device is a DIMM, this specifies the full DQ width of the DIMM. If the interface is composed of discrete components, this specifies the DQ width of each discrete component.) • Data DQ Width. • ECC DQ Width. • Die DQ Density. • CS Width. • Memory Speed Bin. • Memory Operating Frequency.
PHY	Select the desired mode to connect the EMIF IP to user logic: • Synchronous fabric. • Asynchronous fabric. • Reference clock • NoC mode. • A/C placement • Pin swizzle map • Enable the Debug Toolkit
Advanced: Memory Timing	Allows you to modify the timing values.
Advanced: Analog Overrides	Allows you to modify the termination, and V_{REF} settings and drive strengths.
Example Design	The **Example Design** tab lets you select which HDL to use for the top-level files, and which file sets you want the design example to generate. You can also enable the performance monitor and modify the length traffic for the traffic generator. You should make these selections before clicking the **Generate Example Design...** button. The generated design example is a complete EMIF system consisting of the EMIF IP and a driver that generates random traffic to validate the memory interface.

Note: When using banks 2A and 2B, select **Ch0 Bot Sub-Bank/Ch1 Bot Sub-Bank** for the **AC placement** address and command placement parameter. For banks 3A/3B, 3C/3D, or 2C/2D, there are two options available for the **AC placement** parameter:

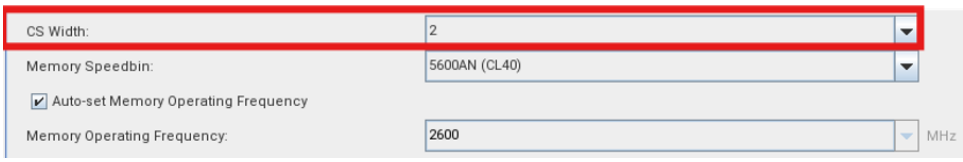
- You must select **Ch0 Top Sub-Bank/Ch1 Bot Sub-Bank** when placing CH0 address and command pins in the top sub-bank.
- You must select **Ch0 Bot Sub-Bank/Ch1 Top Sub-Bank** when placing CH0 address and command pins in the bottom sub-bank.

Figure 6. EMIF Parameter Editor AC Placement Options



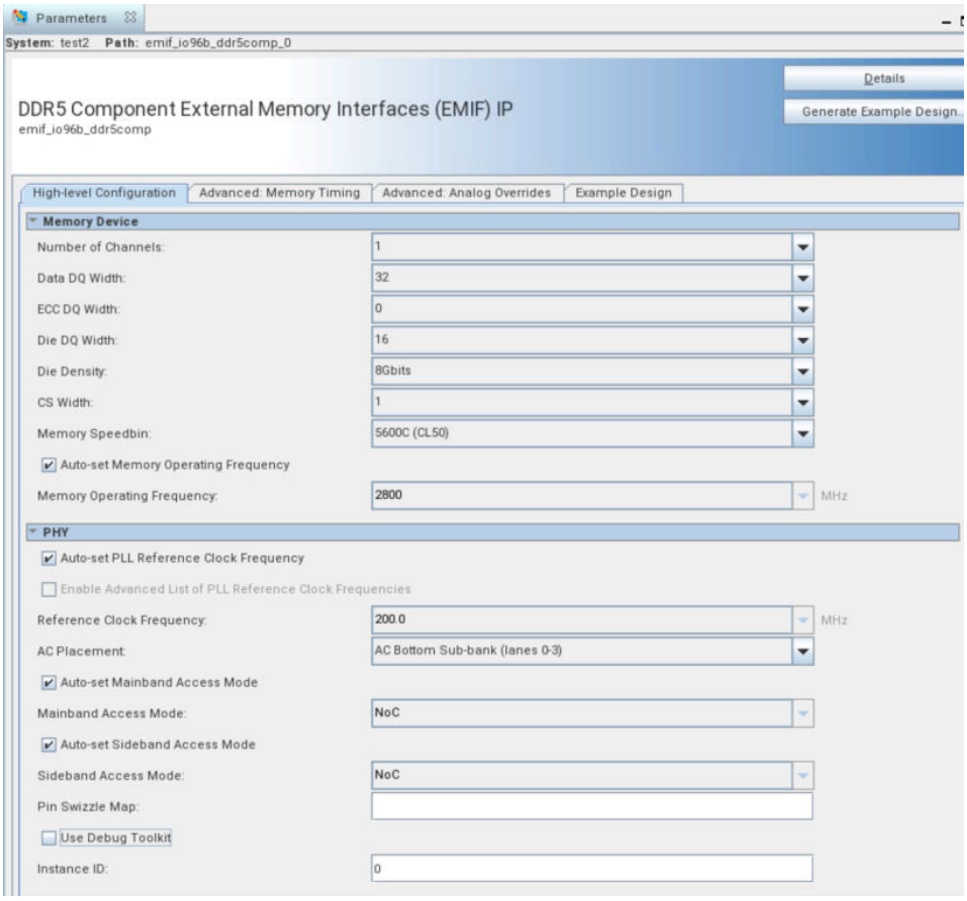
When using DIMMs with dual-die components, ensure that you change the CS Width field to 2.

Figure 7. Command-Address Mirroring



For additional information, refer to the *DIMM Support* subsection of the *DDR5 Data Width Mapping* topic in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.

Figure 8. External Memory Interfaces IP Parameter Editor



For detailed information on individual parameters, refer to the appropriate protocol-specific chapter in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.

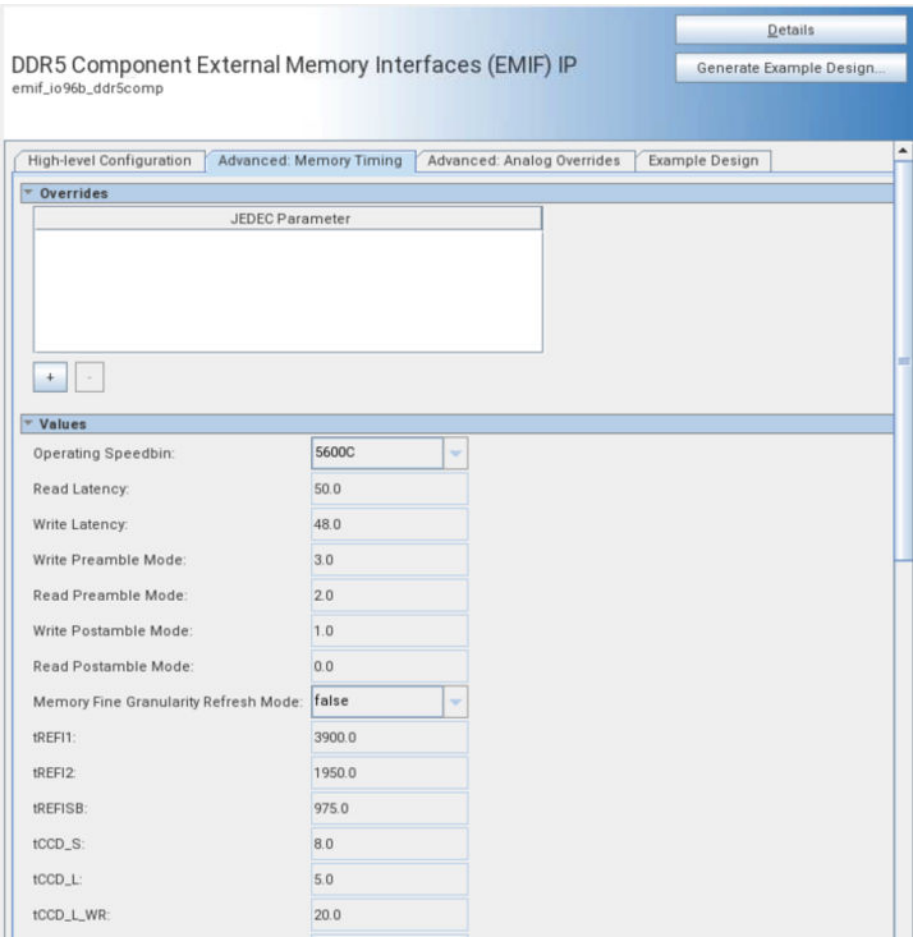
2.2.2. Using Custom Memory Timing and I/O Configuration

If you want, you can use custom memory device timing and analog configurations by modifying the values on the **Advanced: Memory timing** and **Advanced Analog: overrides** tabs. .

To modify the **Advanced: Memory timing** and **Advanced Analog: overrides** values, follow these steps:

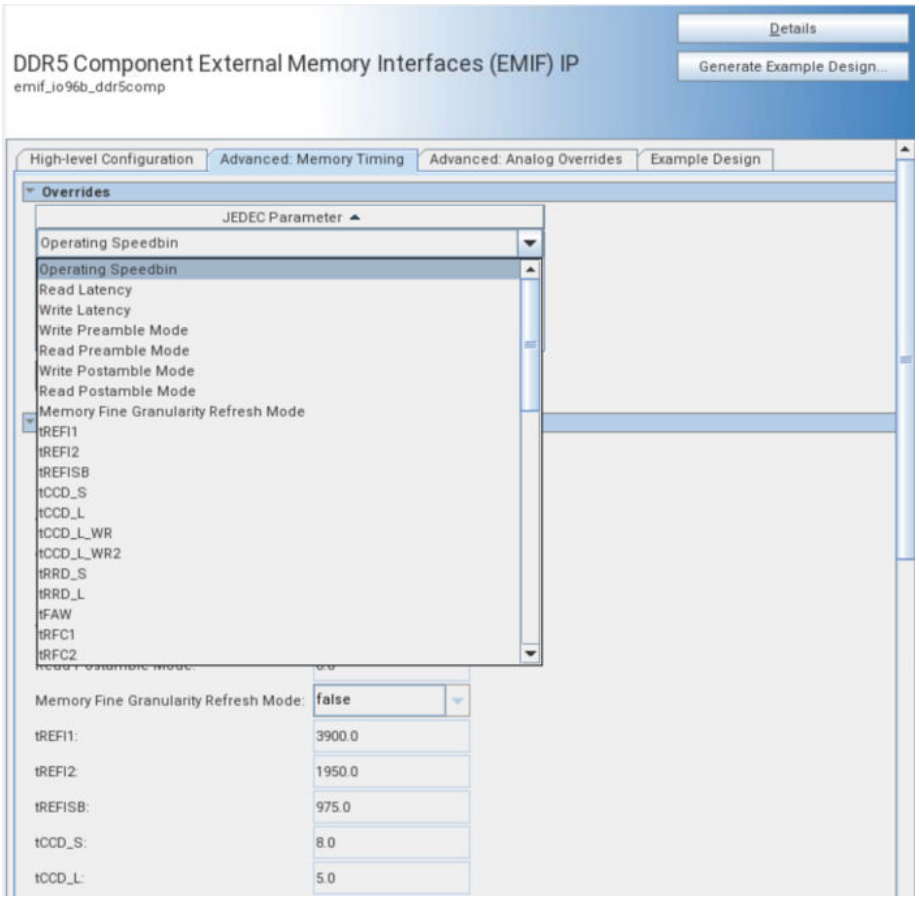
1. In the IP parameter editor, select the **Advanced: Memory timing** or **Advanced Analog: overrides** tab.
2. In the overrides tab, click the **+** beneath the **JEDEC Parameter** box.

Figure 9. Advanced: Memory Timing Tab



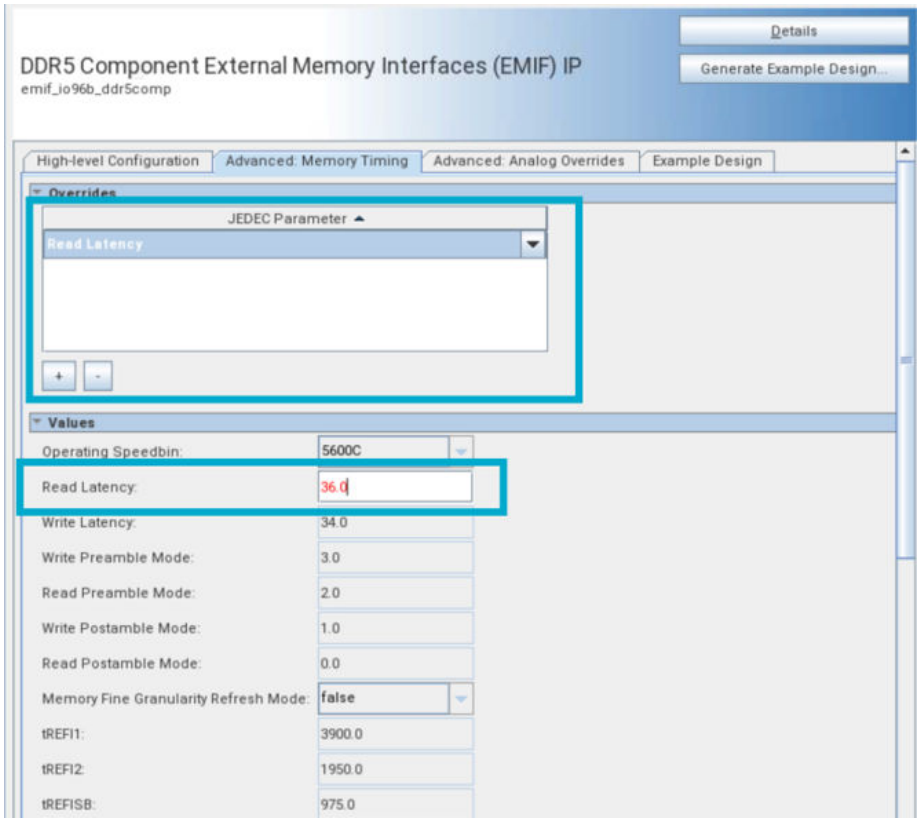
3. Click the plus sign (+) that appears below the **JEDEC Parameter** box. A drop-down menu appears. Select the parameter that you want to change.

Figure 10. Drop-down Menu



4. In the list under the values section, search for the parameter you want to change and input the value on the field next to it.

Figure 11. Modifying an Advanced Memory Timing Value



If the specified value is not valid for the parameter, an error message appears, listing the valid values.

2.3. Parameterizing the External Memory Interface for HPS IP

This section describes launching the EMIF IP for HPS and provides guidance for parameterizing the IP.

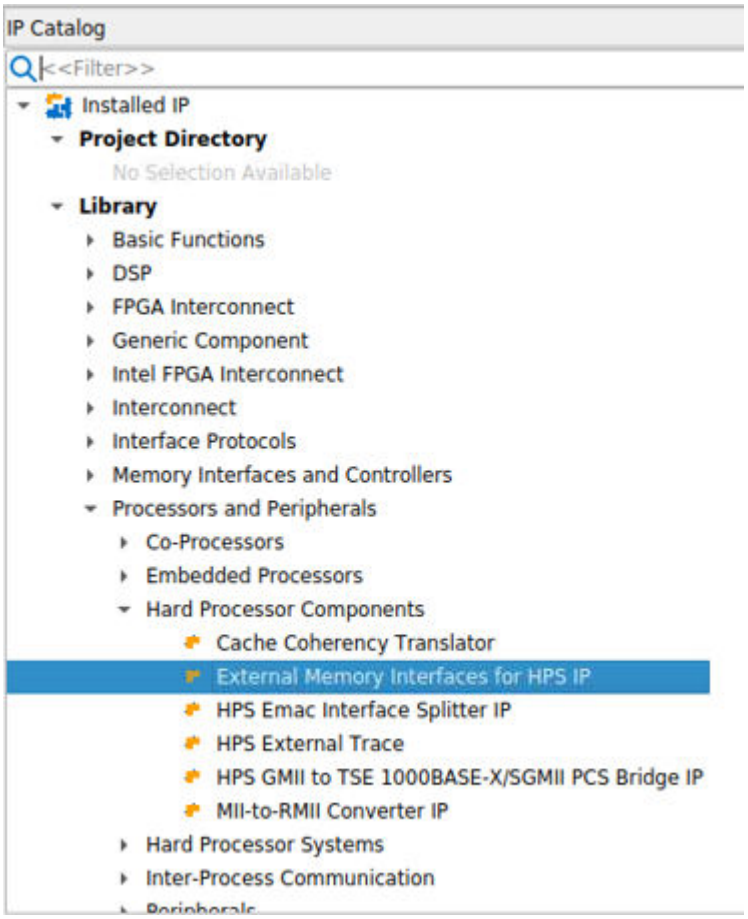
To create your EMIF HPS IP, you first launch the Platform Designer and then search for the *External Memory Interfaces for HPS IP* in the IP Catalog.

To launch the External Memory Interfaces for HPS IP, follow these steps:

1. Create a Quartus Prime project and select an Agilex 7 M-Series device.
2. Go to the Platform Designer and create a system.
3. Click **IP catalog ➤ Processors and Peripherals ➤ Hard Processor Components ➤ External Memory Interfaces for HPS IP**.

The preceding steps open the External Memory Interfaces for HPS IP parameter editor, where you can set high-level parameters for your interface.

Figure 12. Launching the External Memory Interfaces for HPS IP Parameter Editor



Click to instantiate the External Memory Interfaces for HPS IP in your Platform Designer system.

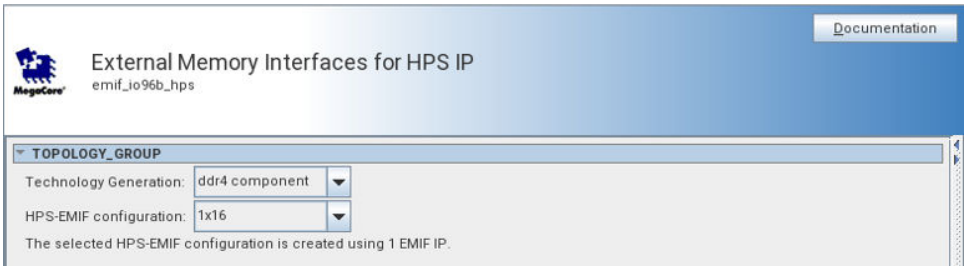
2.3.1. Parameterization Flow

To parameterize the EMIF for HPS IP, you must first configure the high-level topology by defining the following options:

- Memory protocol for the EMIF IP
- Memory configuration

After creating and generating the desired high-level topology, you can set more specific parameters and features of the IP by clicking **Dive into Packaged Subsystem**. The following figure shows the parameter editor interface.

Figure 13. External Memory Interfaces for HPS IP Parameter Editor



2.3.2. Parameterizing the EMIF HPS IP

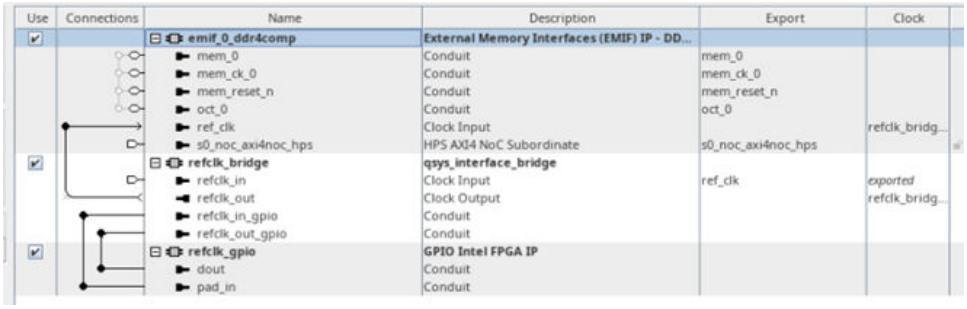
After you have configured the high-level topology, you can continue parameterizing the EMIF IP for the selected memory protocol.

To parameterize the EMIF IP, follow these steps:

1. Click **Dive Into Packaged Subsystem**. A new Platform Designer window appears, listing the available EMIF IP.
2. In the new Platform Designer window, set the desired parameters for your EMIF IP and selected memory protocol.

The following figure depicts the Platform Designer.

Figure 14. Example of the Platform Designer Window to Parametrize the External Memory Interfaces for HPS IP

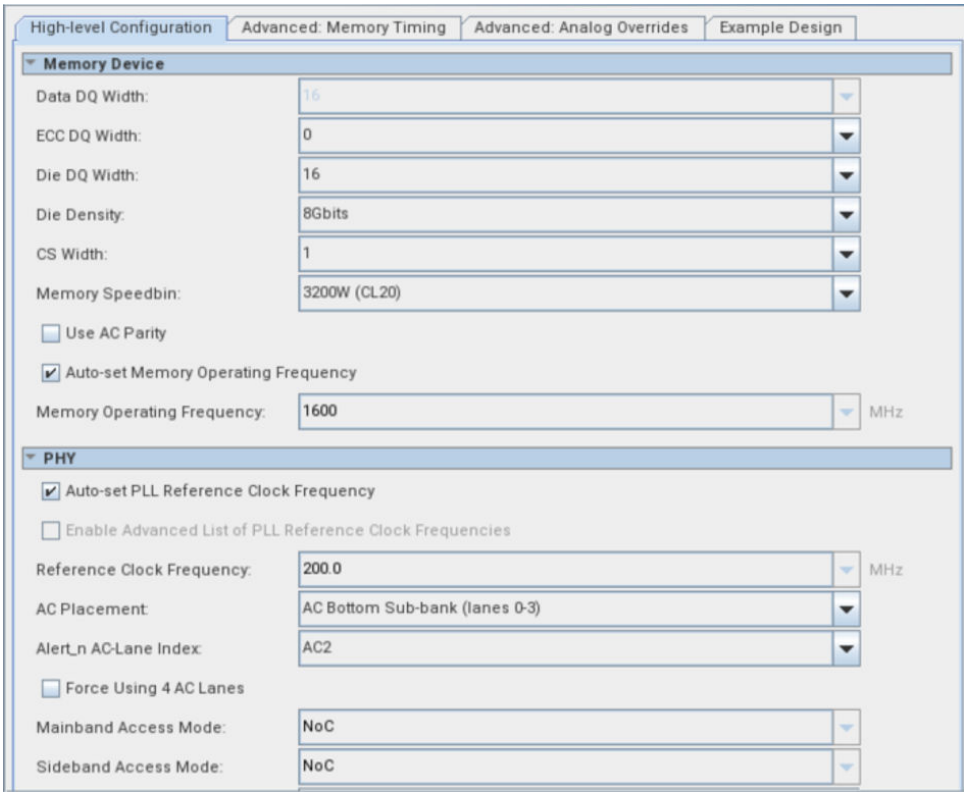


2.3.3. Parameterizing the EMIF IP

Click the EMIF IP that you want to parameterize.

The **Parameters** tab shows the selected IP and all the parameters that you can change.

Figure 15. External Memory Interfaces Parameters Tab



For more information on the possible parameters and options, refer to the appropriate section describing the selected memory protocol in the *External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide*. For information on how to parametrize the EMIF IP, refer to *Generating and Configuring the EMIF IP*, in this document.

Note: In External Memory Interface for HPS IP dual channel configurations, CH0 must be assigned to bank 3D and CH1 to bank 3C.

2.4. Configuring DQ Pin Swizzling

It is important to strictly follow the pin placement for a given memory topology when assigning pin locations for your EMIF IP.

Do not change the location for the EMIF pin using a `.qsf` assignment or the pin planner if you need to swap the DQ pins within a DQS group or the DQS group to simplify board design. For example, if you implement a x32 DDR4 interface, the EMIF pin location must adhere to the *x32* column in the *DDR4 Pin Placement* table in the *Product Architecture* chapter of the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide. There is no flexibility to swap the address command pin location.

The following tables summarize the parameters for pin swizzling and byte swizzling, respectively.

Table 2. Swizzling Parameters for the Pin Swizzle Map

Parameter	Description
PIN_SWIZZLE_CH<*m*>_DQS<*n*>	Used for swizzling DQ pin within DQS group <*n*> for channel <*m*>. *m*=0 only for DDR4.
PIN_SWIZZLE_CH<*m*>_ECC	Used for swizzling DQ pin within the following lane: • ECC (When side-band ECC is enabled) • RUSER/WUSER lane (when **Extra DQ Byte Lane** is enabled) *m*=0 only for DDR4
PIN_SWIZZLE_PRI_DQS<*n*>	Used for swizzling DQ pin within DQS group <*n*> in the primary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.
PIN_SWIZZLE_SEC_DQS<*n*>	Used for swizzling DQ pin within DQS group <n> in the secondary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.
PIN_SWIZZLE_PRI_ECC	Used for swizzling DQ pin within the ECC lane or RUSER/WUSER lane (when the **Extra DQ Byte Lane** is enabled), if this lane is implemented in the primary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.
PIN_SWIZZLE_SEC_ECC	Used for swizzling DQ pin within the ECC lane or RUSER/WUSER lane (when the **Extra DQ Byte Lane** is enabled), if this lane is implemented in the secondary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.

For device widths of x4 and x8, you can swizzle each DQ pin within its DQS group. For device widths of x16, you can swizzle each DQ pin within the lower byte and upper byte respectively. You cannot swizzle DQ pin for the lower byte to upper byte and vice versa.

Table 3. Byte Swizzling Parameters for the Pin Swizzle Map

Parameter	Description
BYTE_SWIZZLE_CH<*m*>	Used for swizzling DQS group for CH<*m*> of the interface. *m*=0 for DDR4.
BYTE_SWIZZLE_PRI	Used for swizzling DQS group in the primary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.
BYTE_SWIZZLE_SEC	Used for swizzling DQS group in the secondary IO96 bank. This parameter is only applicable for lockstep configuration implemented with 2 adjacent IO96 banks.

Related Information

Pin Swizzling Tool

2.4.1. Example: DQ Pin Swizzling Within DQS group for x32 DDR4 interface

Example: DQ Pin Swizzling Within DQS group for x32 DDR4 interface

This example uses the lane placement of the following table, using a memory device of x8 width.

Table 4. Lane Placement for a x32 DDR4 Interface

Lane Number	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
Default placement	DQ[0]	AC0	AC1	AC2	DQ[1]	DQ[2]	DQ[3]	GPIO

Table 5. Example of Swizzling DQ pin in BL0 (DQS group 0)

Lane	Pin Index	DDR4 x32 (Default Placement)	After Swizzling / Swapping
BL0	11	MEM_DQ[7]	MEM_DQ[6]
	10	MEM_DQ[6]	MEM_DQ[5]
	9	MEM_DQ[5]	MEM_DQ[7]
	8	MEM_DQ[4]	MEM_DQ[4]
	7		
	6	MEM_DM_N[0]	MEM_DM_N[0]
	5	MEM_DQS_C[0]	MEM_DQS_C[0]
	4	MEM_DQS_T[0]	MEM_DQS_T[0]
	3	MEM_DQ[3]	MEM_DQ[0]
	2	MEM_DQ[2]	MEM_DQ[2]
	1	MEM_DQ[1]	MEM_DQ[1]
	0	MEM_DQ[0]	MEM_DQ[3]

To achieve this swizzling in DQS group 0, you must enter `PIN_SWIZZLE_CH0_DQS0=3,1,2,0,4,7,5,6;` in the **Pin Swizzle Map** field in the **PHY** section of the **High-level Configuration** tab in the EMIF IP parameter editor.

Figure 17. Entering a PIN_SWIZZLE Specification

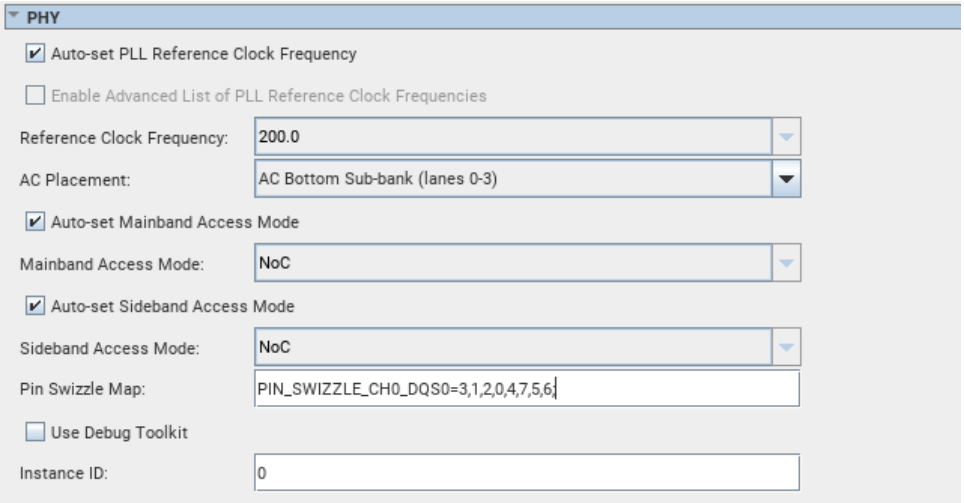
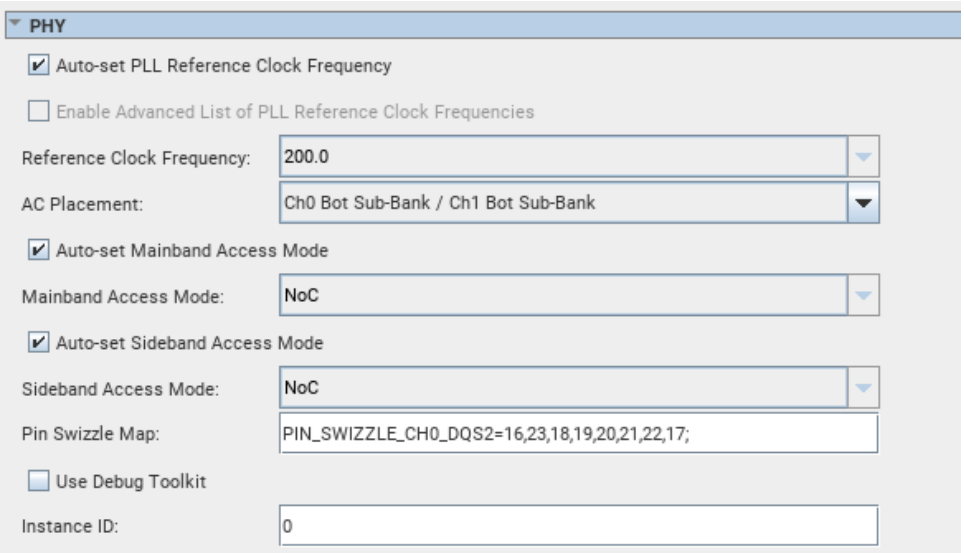


Table 6. Example of Swizzling DQ pin in BL5 (DQS group 2)

Lane	Pin Index	DDR4 x32 (from table)	After Swizzling / Swapping
BL5	71	MEM_DQ[23]	MEM_DQ[17]
	70	MEM_DQ[22]	MEM_DQ[22]
	69	MEM_DQ[21]	MEM_DQ[21]
	68	MEM_DQ[20]	MEM_DQ[20]
	67		
	66	MEM_DM_N[2]	MEM_DM_N[2]
	65	MEM_DQS_C[2]	MEM_DQS_C[2]
	64	MEM_DQS_T[2]	MEM_DQS_T[2]
	63	MEM_DQ[19]	MEM_DQ[19]
	62	MEM_DQ[18]	MEM_DQ[18]
	61	MEM_DQ[17]	MEM_DQ[23]
	60	MEM_DQ[16]	MEM_DQ[16]

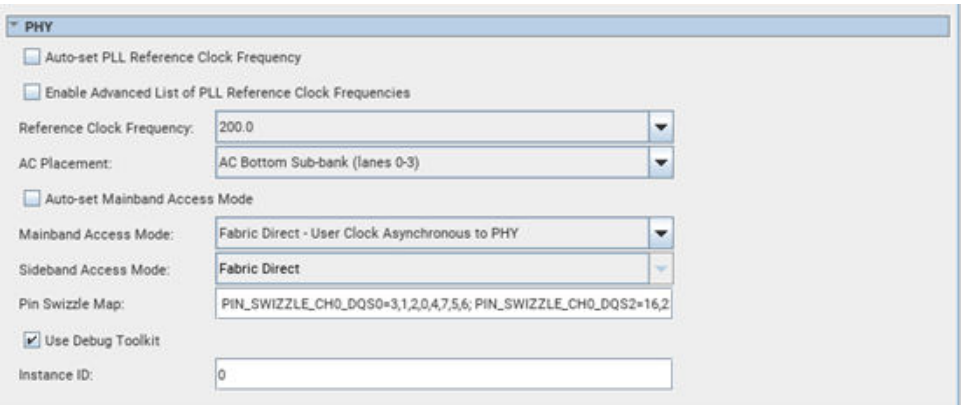
To achieve this swizzling in DQS Group 2, you must enter `PIN_SWIZZLE_CH0_DQS2=16,23,18,19,20,21,22,17;` in the **Pin Swizzle Map** field of the **PHY** section of the **High-level Configuration** tab in the parameter editor.

Figure 18. Entering a PIN_SWIZZLE Specification



You can enter multiple specifications in the **Pin Swizzle Map** field, each separated by a comma.

Figure 19. Entering Multiple PIN_SWIZZLE Specifications



2.4.2. Example: Byte Swizzling for a x32 DDR4 interface, using a memory device of x8 width

Example: Byte Swizzling for a x32 DDR4 interface, using a memory device of x8 width

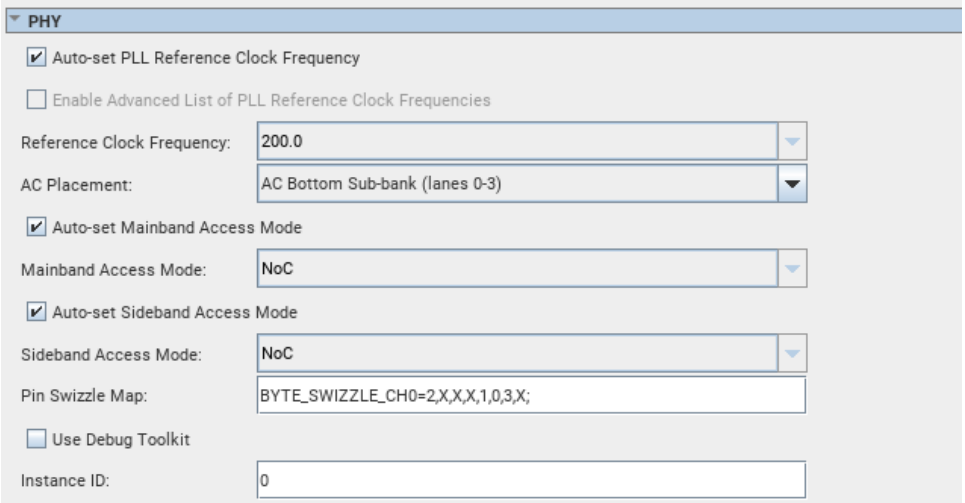
Table 7.

Lane Number	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
Default Placement	DQ[0]	AC[0]	AC[1]	AC[2]	DQ[1]	DQ[2]	DQ[3]	GPIO
After Byte Swizzling	DQ[2]	AC[0]	AC[1]	AC[2]	DQ[1]	DQ[0]	DQ[3]	GPIO
BYTE SWIZZLE	2	X	X	X	1	0	3	X

This example illustrates swizzling DQS group 0 with DQS group 2. The BYTE_SWIZZLE_CH0 denotes the DQS group implemented in the lane after the swizzling. *X* indicates that the lane is not used as a data lane (meaning that it serves as an address/command lane or is not used by the EMIF interface).

To achieve this swizzling, you must enter `BYTE_SWIZZLE_CH0=2,X,X,X,1,0,3,X;` in the **Pin Swizzle Map** field in the **PHY** section of the **High-level Configuration** tab in the parameter editor.

Figure 20. Swizzling DQS group 0 with DQS group 2



2.4.3. Combining Pin and Byte Swizzling

Example : Combining Pin and Byte Swizzling

This example combines the two previous examples, entering the following swizzle parameters:

- `PIN_SWIZZLE_CH0_DQS0=3,1,2,0,4,7,5,6;`
- `PIN_SWIZZLE_CH0_DQS2=16,23,18,19,20,21,22,17;`
- `BYTE_SWIZZLE_CH0=2,X,X,X,1,0,3,X;`

The following table shows the resulting pin placement after DQ pin and byte swizzling.

Table 8. Example of resulting pin placement after DQ pin and byte swizzling

Lane	Pin Index	Default	Effective Pinout
BL5	71	MEM_DQ[23]	MEM_DQ[6]
	70	MEM_DQ[22]	MEM_DQ[5]
	69	MEM_DQ[21]	MEM_DQ[7]
	68	MEM_DQ[20]	MEM_DQ[4]
	67		
	66	MEM_DM_N[2]	MEM_DM_N[0]
	65	MEM_DQS_C[2]	MEM_DQS_C[0]
	64	MEM_DQS_T[2]	MEM_DQS_T[0]
	63	MEM_DQ[19]	MEM_DQ[0]
	62	MEM_DQ[18]	MEM_DQ[2]
	61	MEM_DQ[17]	MEM_DQ[1]
	60	MEM_DQ[16]	MEM_DQ[3]
BL0	11	MEM_DQ[7]	MEM_DQ[17]
	10	MEM_DQ[6]	MEM_DQ[22]
	9	MEM_DQ[5]	MEM_DQ[21]
	8	MEM_DQ[4]	MEM_DQ[20]
	7		
	6	MEM_DM_N[0]	MEM_DM_N[2]
	5	MEM_DQS_C[0]	MEM_DQS_C[2]
	4	MEM_DQS_T[0]	MEM_DQS_T[2]
	3	MEM_DQ[3]	MEM_DQ[19]
	2	MEM_DQ[2]	MEM_DQ[18]
	1	MEM_DQ[1]	MEM_DQ[23]
	0	MEM_DQ[0]	MEM_DQ[16]

Note: When using the EMIF IP with 2 channels , the syntax for CH0 and CH1 must be the following:

- `PIN_SWIZZLE_CH0_DQS0/ PIN_SWIZZLE_CH1_DQS0`
- `PIN_SWIZZLE_CH0_DQS2/ PIN_SWIZZLE_CH1_DQS2`
- `BYTE_SWIZZLE_CH0/ BYTE_SWIZZLE_CH1`

2.4.4. Example: Swizzling for a x32 + ECC interface

Example: Swizzling for a x32 + ECC interface

Table 9.

Scheme	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
DDR4_AC_TOP	DQ[4]	DQ[3]	DQ[2]	DQ[1]	AC1	AC2	AC0	sDQ[0]
DQS group number in byte swizzling notation	3	2	1	0	AC1	AC2	AC0	ECC
After Byte Swizzling	3	2	0	1	X	X	X	ECC

In this example, BL7 cannot be swapped with other used DQS group. It is used as follows:

- RUSER/WUSER Lane in x40 configuration
- ECC Lane in x32 + ECC configuration

This example illustrates swizzling DQS group 1 (BL3) with DQS group 0 (BL2). To achieve this swizzling, enter the following `BYTE_SWIZZLE_CH0` specification in the **Pin Swizzle Map**:
`BYTE_SWIZZLE_CH0=3,2,0,1,X,X,X,ECC;`

In DDR4 x32 + ECC configuration, the highest index DQS group is used as ECC lane. We use PIN_SWIZZLE_CH0_ECC for swizzle the DQ pins within the ECC lane in this case. Note that the valid value for pin swizzling specification in the ECC lane is always 0-7 only.

Table 10. Example of DQ pin swizzling in ECC lane for a DDR4 x32 + ECC interface

Lane	Pin Index	Default Placement	After Swizzling
BL7	95	MEM_DQ[39]	MEM_DQ[36]
	94	MEM_DQ[38]	MEM_DQ[37]
	93	MEM_DQ[37]	MEM_DQ[38]
	92	MEM_DQ[36]	MEM_DQ[39]
	91		
	90	MEM_DM_N[4]	MEM_DM_N[4]

continued...

Lane	Pin Index	Default Placement	After Swizzling
	89	MEM_DM_C[4]	MEM_DM_C[4]
	88	MEM_DM_T[4]	MEM_DM_T[4]
	87	MEM_DQ[35]	MEM_DQ[34]
	86	MEM_DQ[34]	MEM_DQ[35]
	85	MEM_DQ[33]	MEM_DQ[32]
	84	MEM_DQ[32]	MEM_DQ[33]

To achieve the pin swizzling shown in the above table, enter the following BYTE_SWIZZLE_CH0 specification in **Pin Swizzle Map**:
`PIN_SWIZZLE_CH0_ECC=1,0,3,2,7,6,5,4;`

2.4.5. Example: Swizzling for a 2Ch x32 + ECC interface

Table 11. Byte Swizzling for an RDIMM DDR5 2Ch x 32 + ECC Interface Implemented with Ch0 Top Sub-Bank / Ch1 Bottom Sub-Bank

Scheme	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
Ch 0								
Default Placement	GPIO	DQ[ECC]	DQ[3]	DQ[2]	AC0	AC1	DQ[0]	DQ[1]
DQS Group Number in Byte Swizzling Notation	X	ECC	3	2	X	X	0	1
After Byte Swizzling	X	ECC	3	2	X	X	1	0
Ch 1								
Default Placement	DQ[1]	DQ[0]	AC0	AC1	DQ[2]	DQ[3]	DQ[ECC]	GPIO
DQS Group Number in Byte Swizzling Notation	1	0	X	X	2	3	ECC	X
After Byte Swizzling	1	0	X	X	3	2	ECC	X

This example illustrates byte swizzling in Ch0 DQS group 1 (BL7) with DQS group 0 (BL6) and Ch1 DQS group 2 (BL4) with DQS group 3 (BL5). To achieve this swizzling, enter the following `BYTE_SWIZZLE_CH0` and `BYTE_SWIZZLE_CH1` specification in the **Pin Swizzle Map**:

- `BYTE_SWIZZLE_CH0=X,ECC,3,2,X,X,1,0;`
- `BYTE_SWIZZLE_CH1=1,0,X,X,3,2,ECC,X;`

Table 12. Example of DQ Pin Swizzling in Ch0 ECC Lane for a 2Ch x 32 + ECC Interface

Lane	Pin Index	Default Placement	After Swizzling
BL3	47	MEM_DQ[23]	MEM_DQ[23]
	46	MEM_DQ[22]	MEM_DQ[22]
	45	MEM_DQ[21]	MEM_DQ[20]
	44	MEM_DQ[20]	MEM_DQ[21]
	43		
	42		
	41		
	40		
	39	MEM_DQ[19]	MEM_DQ[16]
	38	MEM_DQ[18]	MEM_DQ[19]
	37	MEM_DQ[17]	MEM_DQ[18]
	36	MEM_DQ[16]	MEM_DQ[17]
BL1	23	MEM_DQ[39]	MEM_DQ[36]
	22	MEM_DQ[38]	MEM_DQ[37]
	21	MEM_DQ[37]	MEM_DQ[38]
	20	MEM_DQ[36]	MEM_DQ[39]
	19		
	18		
	17		
	16		
	15	MEM_DQ[35]	MEM_DQ[34]
	14	MEM_DQ[34]	MEM_DQ[35]
	13	MEM_DQ[33]	MEM_DQ[32]
	12	MEM_DQ[32]	MEM_DQ[33]

To achieve the pin swizzling for the out-of-band controller ECC byte (DQ[39:32]) shown in the above table, enter the following BYTE_SWIZZLE_CH0 specification in the **Pin Swizzle Map**:

```
PIN_SWIZZLE_CH0_ECC=1,0,3,2,7,6,5,4;
```

This is a full ECC byte (8-bit out-of-band controller ECC). In the case where it is implemented with x4 devices it must follow the x4 restrictions of placing the lower 4 bits of the byte in the lower half and the upper four bits of the byte in the upper half.

Note: For 4-bit out-of-band controller ECC implemented with x4, x8 or x16 devices, the 4 ECC pins must be placed on the lower half of the byte lane. In this case, users only need to specify 4 values for the pin swizzle.

Example: `PIN_SWIZZLE_CH0_ECC=1,0,3,2;`

For x4 device widths, the values in BYTE_SWIZZLE represent the bundles of 2 x4 DQS groups. The lower DQS group of the bundle is connected to the lower half of the byte lane, and the upper DQS group of the bundle is connected to the upper half of the byte lane.

To achieve the pin swizzling for DQ[23:16] shown in the table above using x4 devices, enter the following pin swizzling specification:

- `PIN_SWIZZLE_CH0_DQS4=17,18,19,16;`
- `PIN_SWIZZLE_CH0_DQS5=21,20,22,23;`

The placement of each DQS bundle and DQS group in this example, after the byte swizzling, follows the pattern shown below:

- `CH0 Bundle 2 is placed on BL3:`
  - ☐ `CH0 DQS4 connected to lower half of BL3`
  - ☐ `CH0 DQS5 connected to upper half of BL3`

2.4.6. Example: Byte Swizzling for Lockstep Configuration

In lockstep configuration, a DQ lane with a letter *s* prefix is used as RUSER/WUSER or ECC lane. This lane corresponds to the ECC lane in byte-swizzling notation; it cannot be swapped with other DQS lanes.

Case A: DDR4 with sDQ[0]

`sDQ[0]` maps to the RUSER/WUSER group. It corresponds to the ECC lane in byte-swizzling notation. Because the BYTE_SWIZZLE_CH0 specification can only accept 0,1,2,3,ECC and X, the group number for other DQS groups must be reduced by 1.

This example illustrates the swizzling of DQS group 3 (BL0) with DQS group 2 (BL1), and DQS group 1 (BL2) with DQS group 0 (BL3), respectively.

Table 13. Byte Swizzling for Case A

Scheme	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
DDR4/5_AC_TOP	DQ[4]	DQ[3]	DQ[2]	DQ[1]	AC1	Ac2	AC0	sDQ[0]
DQS group number in byte swizzling notation	3	2	1	0	X	X	X	ECC
After Byte Swizzling	2	3	0	1	X	X	X	ECC

To achieve the swizzling described in the above table, enter the following BYTE_SWIZZLE_CH0 specification in the **Pin Swizzle Map**:
`BYTE_SWIZZLE_CH0=2,3,0,1,X,X,X,ECC;`

The method to swizzle the DQ pin within a group is the same for lockstep and non-lockstep configuration. Refer to the *DQ Pin Swizzling Within DQS group for x32 DDR4 interface* example for more information on how to configure DQ Pin Swizzling.

Case B: DDR4 with sDQ[4]

`sDQ[4]` maps to RUSER/WUSER group. It corresponds to the ECC lane in byte-swizzling notation.

Table 14. Byte Swizzling for Case B

Scheme	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
DDR4/5_AC_BOT	DQ[0]	AC0	AC1	AC2	DQ[1]	DQ[2]	DQ[3]	sDQ[4]
DQS group number in byte swizzling notation	0	X	X	X	1	2	3	ECC
After Byte Swizzling	1	X	X	X	0	3	2	ECC

This example illustrates the swizzling of DQS group 0 (BL0) with DQS group 1 (BL4), and DQS group 2 (BL5) with DQS group 3 (BL6), respectively.

To achieve this swizzling, enter the following **BYTE SWIZZLE CH0** specification in the **Pin Swizzle Map**:
`BYTE_SWIZZLE_CH0=1,X,X,X,0,3,2,ECC;`

The method to swizzle the DQ pin within a group is the same for lockstep and non-lockstep configurations. For more information on how to configure DQ pin swizzling, refer to the *DQ Pin Swizzling Within DQS group for x32 DDR4 interface* example.

Case C: DDR4 x72 Lockstep Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

`sDQ[0]` maps to RUSER/WUSER group. It corresponds to the ECC lane in byte-swizzling notation.

Table 15. Byte Swizzling for DDR4 x72 Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
	Primary							
Default Placement	DQ[4]	DQ[3]	DQ[2]	DQ[1]	AC1	AC2	AC0	sDQ[0]
DQS group number in byte swizzling notation	3	2	1	0	X	X	X	ECC
After Byte Swizzling	3	2	0	1	X	X	X	ECC
	Secondary							

continued...

	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
Default Placement	DQ[8]	DQ[7]	DQ[6]	DQ[5]	GPIO	GPIO	GPIO	GPIO
DQS group number in byte swizzling notation	7	6	5	4	X	X	X	X
After Byte Swizzling	6	7	4	5	X	X	X	X

In the above table, the *After Byte Swizzling* rows denote the DQS group implemented in the lane after the swizzling. To achieve the swizzling, enter the following parameter in the **Pin Swizzle Map**:

```
BYTE_SWIZZLE_PRI=3,2,0,1,X,X,X,ECC;
BYTE_SWIZZLE_SEC=6,7,4,5,X,X,X,X;
```

As this configuration is implemented using two IO96 banks, and the ECC lane is placed on the primary IO96 bank, specify the pin swizzling information for the ECC lane using **PIN_SWIZZLE_PRI_ECC**, if you want to swizzle the DQ pins within the ECC lane.

Case D: DDR4 x72 Lockstep Implemented with AC Primary Bot Sub-Bank / Secondary DQ Bot Sub-Bank

Table 16. Byte Swizzling for DDR4 x72 Implemented with AC Primary Bot Sub-Bank / Secondary DQ Bot Sub-Bank

	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
	Primary							
Default Placement	DQ[0]	AC0	AC1	AC2	DQ[1]	DQ[2]	DQ[3]	sDQ[4]
DQS group number in byte swizzling notation	0	X	X	X	1	2	3	ECC
After Byte Swizzling	0	X	X	X	1	3	2	ECC
	Secondary							
Default Placement	DQ8	DQ7	DQ6	DQ5	GPIO	GPIO	GPIO	GPIO
DQS group number in byte swizzling notation	7	6	5	4	X	X	X	X
After Byte Swizzling	7	6	4	5	X	X	X	X

In the above table, the *After Byte Swizzling* rows denote the DQS group implemented in the lane after the swizzling. To achieve the swizzling, enter the following parameter in the **Pin Swizzle Map**:

```
BYTE_SWIZZLE_PRI=0,X,X,X,1,3,2,ECC;
BYTE_SWIZZLE_SEC=7,6,4,5,X,X,X,X;
```

As this configuration is implemented using two IO96 banks, and the ECC lane is placed on the primary IO96 bank, specify the pin swizzling information for the ECC lane using **PIN_SWIZZLE_PRI_ECC**, if you want to swizzle the DQ pins within the ECC lane.

Case E: DDR4 x72 Lockstep Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

Table 17. Byte Swizzling for DDR4 x72 Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
	Primary							
Default Placement	DQ[3]	DQ[2]	DQ[1]	DQ[0]	AC1	AC2	AC0	AC3
DQS group number in byte swizzling notation	3	2	1	0	X	X	X	X
After Byte Swizzling	2	3	0	1	X	X	X	X
	Secondary							
Default Placement	DQ[8]	DQ[7]	DQ[6]	DQ[5]	X	X	GPIO	sDQ[4]
DQS group number in byte swizzling notation	7	6	5	4	X	X	X	ECC
After Byte Swizzling	4	5	6	7	X	X	X	ECC

In the above table, the *After Byte Swizzling* rows denote the DQS group implemented in the lane after the swizzling. To achieve the swizzling, enter the following parameter in the **Pin Swizzle Map**:

```
BYTE_SWIZZLE_PRI=2,3,0,1,X,X,X,X;
BYTE_SWIZZLE_SEC=4,5,6,7,X,X,X,ECC;
```

As this configuration is implemented using two IO96 banks, and the ECC lane is placed on the secondary IO96 bank, specify the pin swizzling information for the ECC lane using **PIN_SWIZZLE_SEC_ECC**, if you want to swizzle the DQ pins within the ECC lane.

Case F: DDR4 x64 Lockstep SODIMM (with x4 Memory Component) Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

Table 18. Byte Swizzling for DDR4 x72 Implemented with AC Primary Top Sub-Bank / Secondary DQ Bot Sub-Bank

	BL0	BL1	BL2	BL3	BL4	BL5	BL6	BL7
	Primary							
Default Placement	DQ0	DQ3	DQ2	DQ1	AC1	AC2	AC0	X
DQS bundle in byte swizzling notation	3	2	1	0	X	X	X	X
After Byte Swizzling	2	3	0	1	X	X	X	X
	Secondary							
Default Placement	DQ7	DQ6	DQ5	DQ4	X	X	X	X
DQS bundle in byte swizzling notation	7	6	5	4	X	X	X	X
After Byte Swizzling	7	6	5	4	X	X	X	X

In the above table, the *After Byte Swizzling* rows denote the DQS bundle implemented in the lane after the swizzling. To achieve the swizzling in the primary IO96 bank, enter the following parameter in the **Pin Swizzle Map**:

```
BYTE_SWIZZLE_PRI= 2,3,0,1,X,X,X,X;
```

For x4 device widths, the values in BYTE_SWIZZLE represent the bundles of 2 x4 DQS groups. The lower DQS group of the bundle is connected to the lower half of the byte lane, while the upper DQS group of the bundle is connected to the upper half of the byte lane.

The placement of each DQS bundle and DQS group in this example, after the byte swizzling, follows the pattern shown below:

- CH0 Bundle 0 is placed on BL2 in the Primary IO96
  - CH0 DQS0 connected to lower half of BL2
  - CH0 DQS1 connected to upper half of BL2
- CH0 Bundle 1 is placed on BL3 in the Primary IO96
  - CH0 DQS2 connected to lower half of BL3
  - CH0 DQS3 connected to upper half of BL3
- CH0 Bundle 2 is placed on BL0 in the Primary IO96
  - CH0 DQS4 connected to lower half of BL0
  - CH0 DQS5 connected to upper half of BL0
- CH0 Bundle 3 is placed on BL1 in the Primary IO96
  - CH0 DQS6 connected to lower half of BL1
  - CH0 DQS7 connected to upper half of BL1
- CH0 Bundle 4 is placed on BL3 in the Secondary IO96
  - CH0 DQS8 connected to lower half of BL3
  - CH0 DQS9 connected to upper half of BL3

In this case, only four values are required to specify the pin swizzling information for a x4 DQS group.

Example:

- PIN_SWIZZLE_PRI_DQS0=3,2,0,1;
- PIN_SWIZZLE_PRI_DQS1=7,6,5,4;
- PIN_SWIZZLE_PRI_DQS6=24,25,26,27;
- PIN_SWIZZLE_PRI_DQS7=28,29,30,31;
- PIN_SWIZZLE_SEC_DQS14=59,57,58,56;
- PIN_SWIZZLE_SEC_DQS15=63,60,61,62;

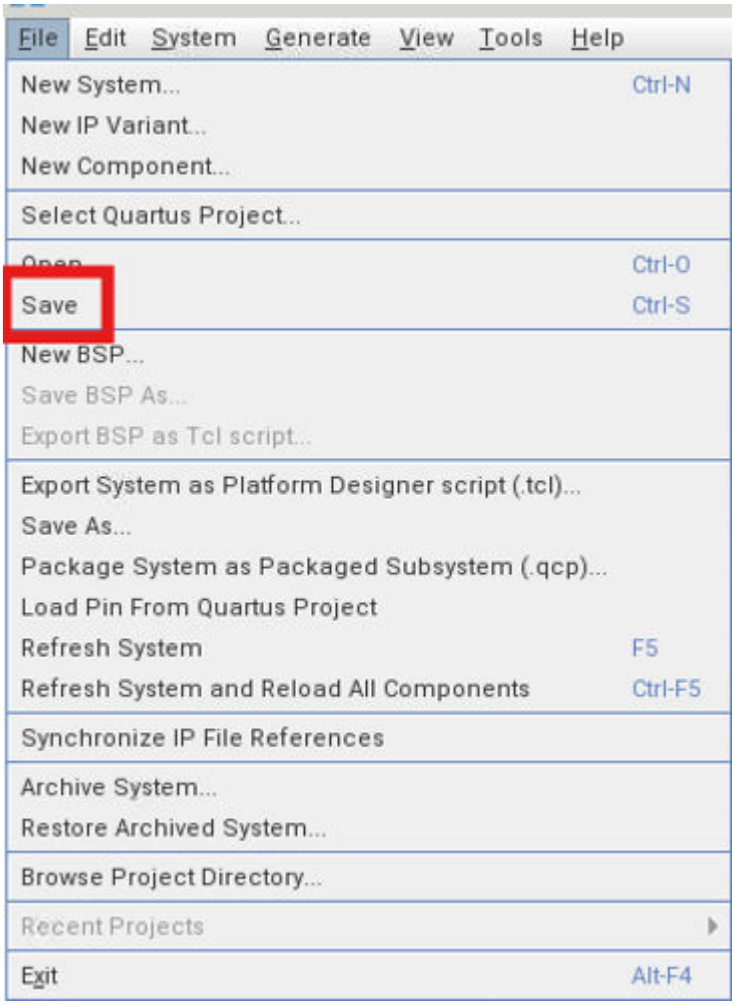
2.5. Generating the Synthesizable EMIF Design Example

For the Agilex 7 M-Series development kit, it is sufficient to leave most of the Agilex 7 M-Series EMIF IP settings at their default values. To generate the synthesizable design example, follow these steps:

1. On the **Example Design** tab, ensure that the **Synthesis** box is set as **True**.

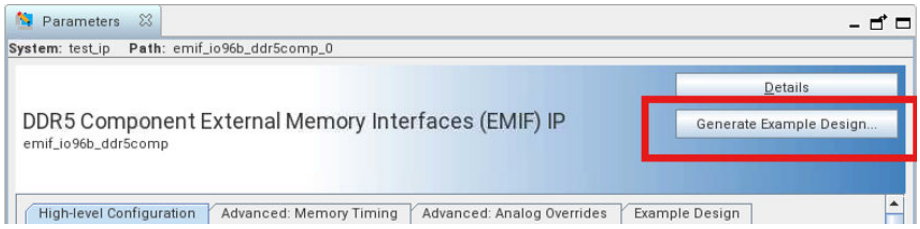
- If you are implementing a single interface example design, configure the EMIF IP and click **File ➤ Save** to save the current settings into the IP variation file (`<user instance name>.ip`).

Figure 21. Saving the Current Settings



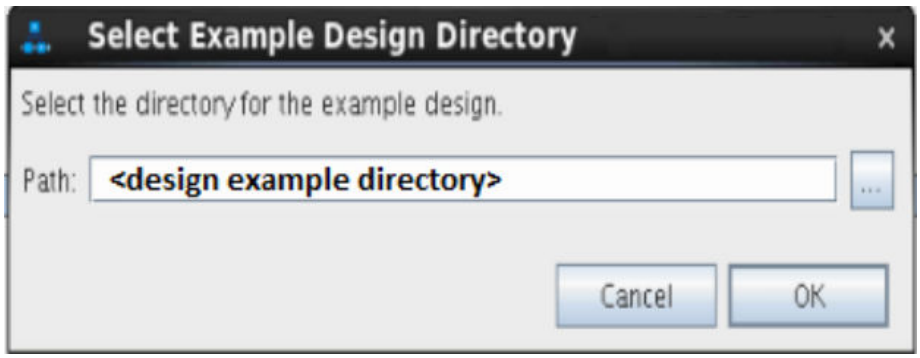
2. Click **Generate Example Design** in the upper-right corner of the window.

Figure 22. Generate Example Design



3. Specify a directory for the EMIF design example and click **OK**. Successful generation of the EMIF design example creates the synthesis file set under a `qii` directory.

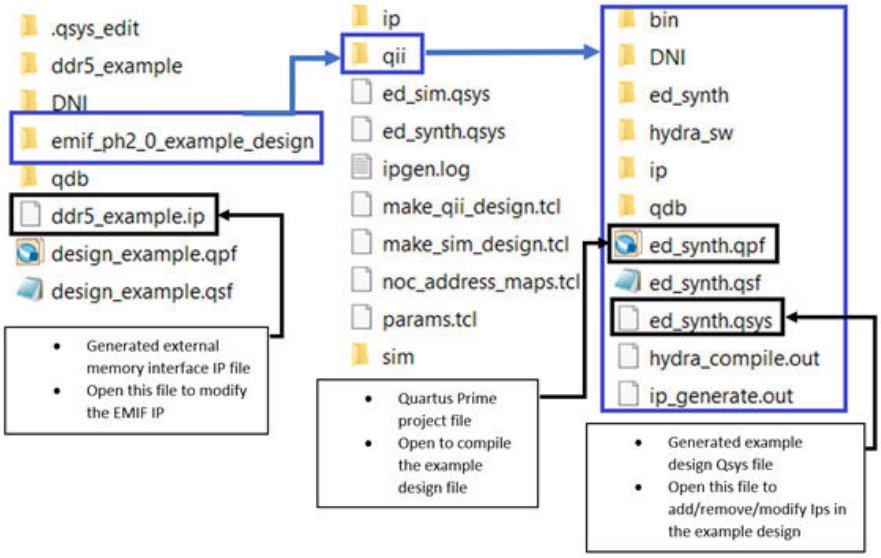
Figure 23. Specifying a Directory



4. Click **File ➤ Exit** to exit the **IP Parameter Editor Pro** window. The system prompts, *Recent changes have not been generated. Generate now?* Click **No** to continue with the next flow.
5. To open the example design, click **File ➤ Open Project**, and navigate to the `<project_directory>/<design_example_name>/qii/ed_synth.qpf` and click **Open**.

   Note: For information on compiling and programming the design example, refer to *Compiling and Programming the Agilex 7 M-Series EMIF Design Example*.

Figure 24. Generated Synthesizable Design Example File Structure



Note:

- For information on constructing a system with two or more external memory interfaces, refer to *Creating a Design Example with Multiple EMIF Interfaces*, in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.
- For information on debugging multiple interfaces, refer to *Enabling the EMIF Toolkit in an Existing Design*, in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.
- If you don't specify *Simulation* or *Synthesis* from the dropdown menu, the destination directory contains only Platform Designer design files, which the Quartus Prime software cannot compile directly, but which you can view or edit in the Platform Designer. In this situation you can run the following commands to generate synthesis and simulation file sets:
  - ☐ To create an Quartus Prime software-compilable project, run the

    ```
    quartus_sh -t make_qii_design.tcl
    ```

    script in the destination directory.
  - ☐ To create a simulation project, run the

    ```
    quartus_sh -t make_sim_design.tcl
    ```

    script in the destination directory.
- If you have generated a design example and then make changes to it in the parameter editor, you must regenerate the design example to see your changes implemented. The newly generated design example does not overwrite the existing design example files.

2.6. Back Annotating Pin Placement and I/O Standard Assignments

To compile a design that instantiates an Agilex 7 M-Series EMIF IP, you must explicitly define a legal placement and I/O standard for for all EMIF pins.

The Quartus Prime Pro Edition software can infer most of the required pin locations and I/O standard assignments if you have properly constrained the following pins:

- Differential P-side reference clock input
- RZQ site
- MEM_RESET_N

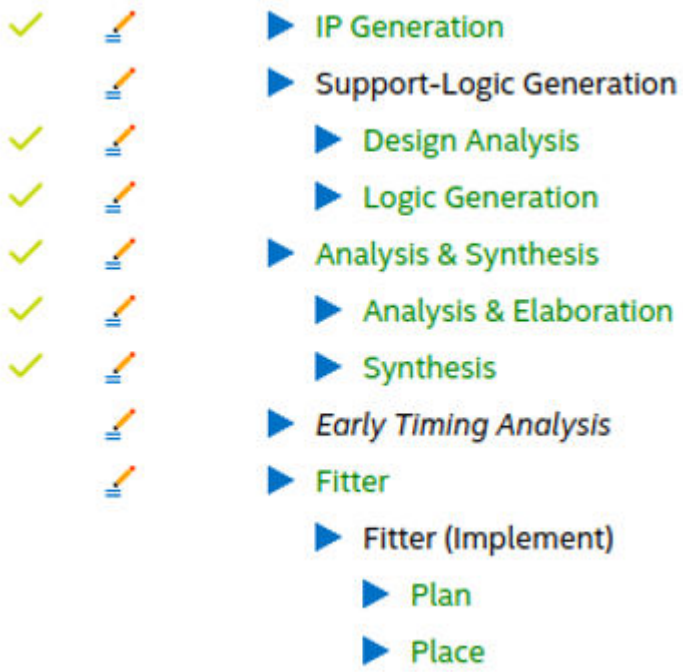
To define pin placement and I/O standards and generate a programming `.sof` file, follow these steps:

1. Open the project's `.qsf` file, and at the end of the file, define a correct placement for RZQ, MEM_RESET_N and the PSide of the reference clock, for example:

```
set_location_assignment PIN_T63  -to ref_clk_clk
set_location_assignment PIN_AC58 -to emif_io96b_0_oct_0_oct_rzqin
set_location_assignment PIN_AB59 -to emif_io96b_0_mem_reset_n_0_mem_reset_n
```

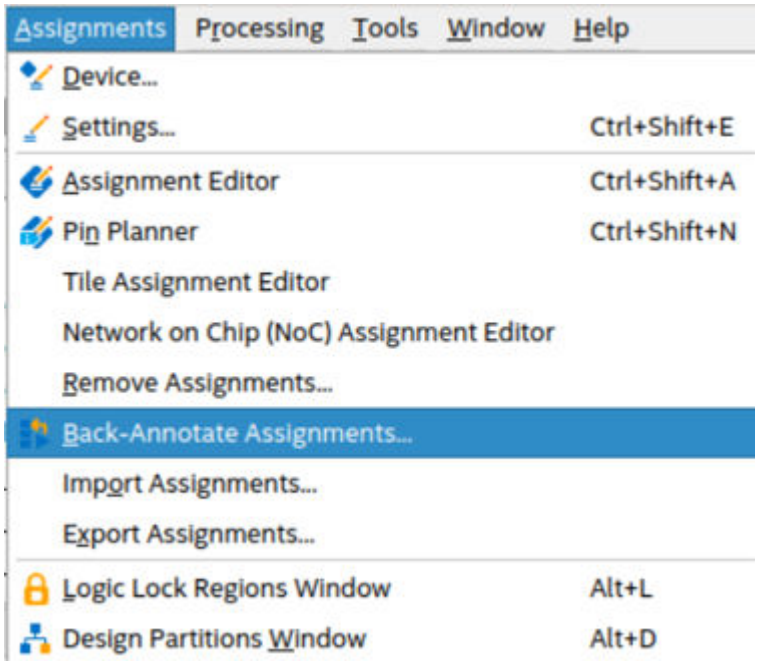
2. Compile your project up to the *placement* stage of the Fitter.

Figure 25. Compilation up to the Placement Stage of the Fitter



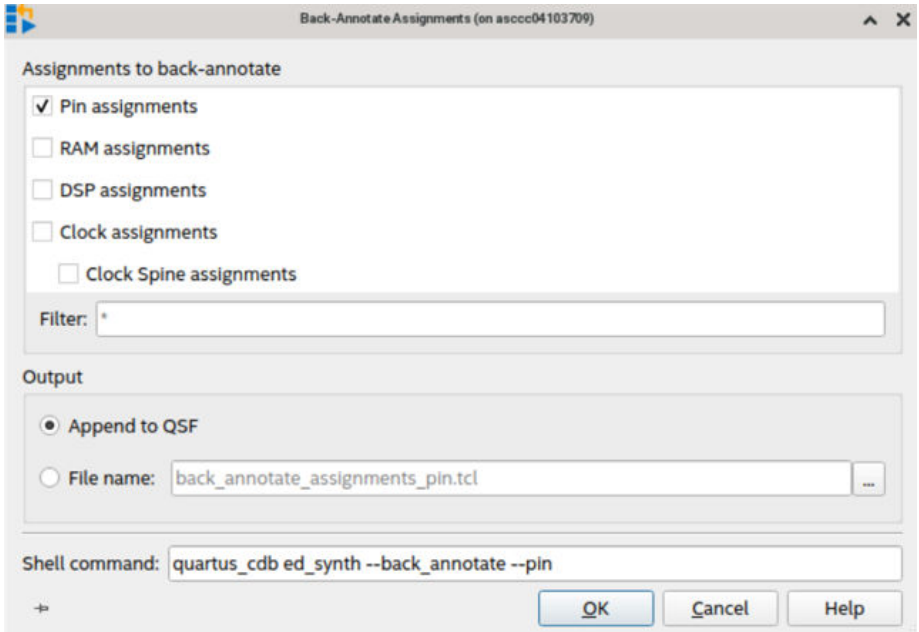
3. Click **Assignments ➤ Back-Annotate Assignments**

Figure 26. Location of Back-Annotate Assignments Tool



4. In the pop-up window, select **Pin Assignments**.
5. In the **Output** field, select **Append to QSF**. Click **OK**.

Figure 27. Back-Annotate Assignments Dialog Box



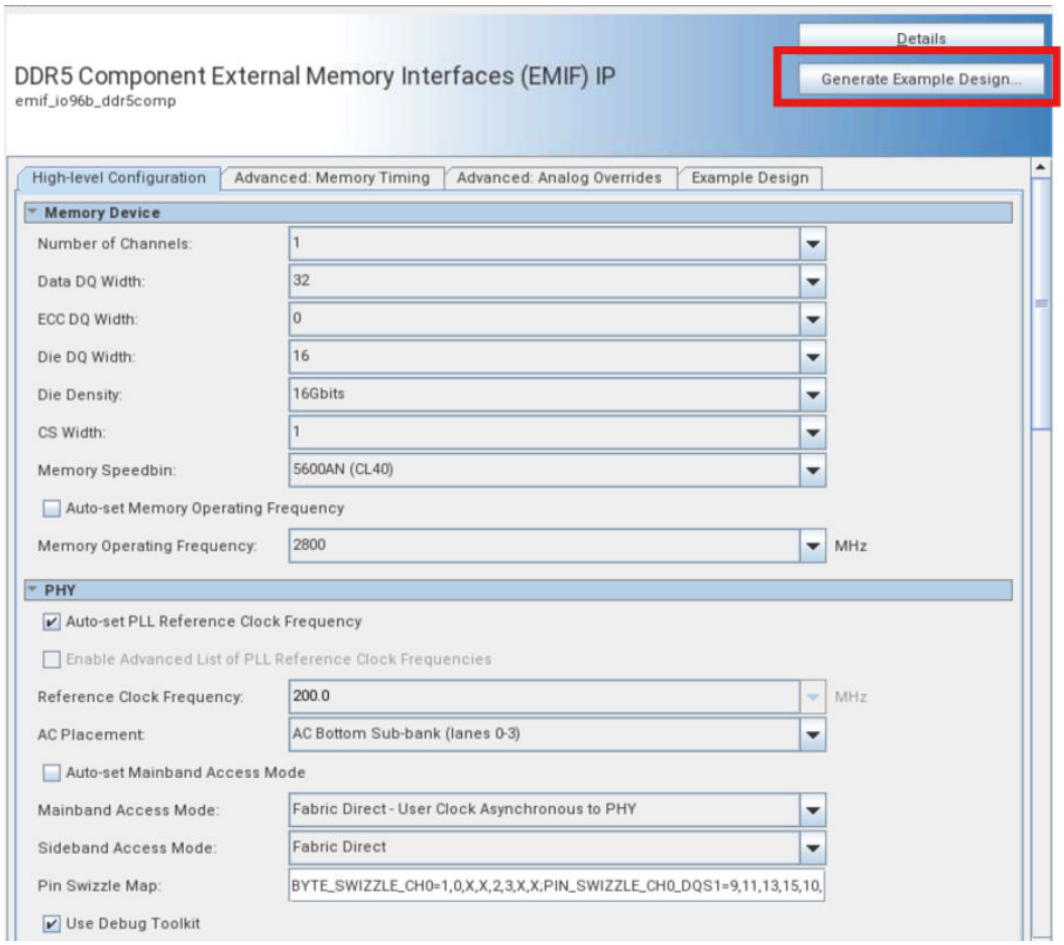
6. Navigate to the Compilation dashboard, and click **Compile Design**.

2.7. Generating the EMIF Design Example for Simulation

For the Agilex 7 M-Series development kit, it is sufficient to leave most of the Agilex 7 M-Series EMIF IP settings at their default values. To generate the design example for simulation, follow these steps:

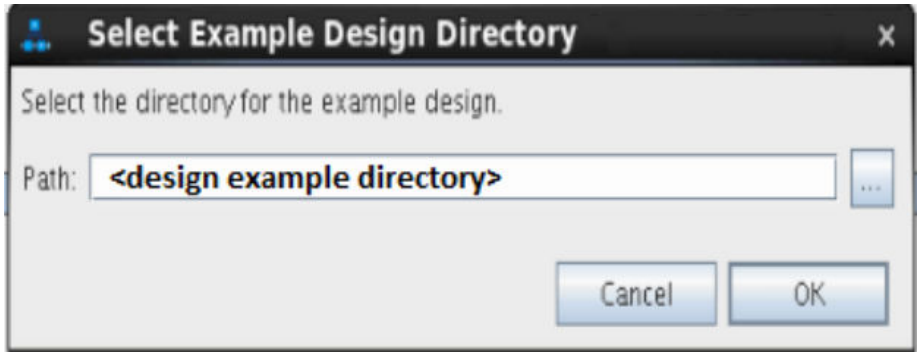
1. On the **Example Designs** tab, ensure that the **Simulation** box is set as **True**. Also choose the required **Simulation HDL format**, either Verilog or VHDL.
2. Configure the EMIF IP and click **File ➤ Save** to save the current setting into the user IP variation file (`<user instance name>.ip`).
3. Click **Generate Example Design** in the upper-right corner of the window.

Figure 28. Generate Example Design



4. Specify a directory for the EMIF design example and click **OK**. Successful generation of the EMIF design example creates multiple file sets for various supported simulators, under a `sim/ed_sim` directory.

Figure 29. Specifying a Directory



5. Click **File ➤ Exit** to exit the **IP Parameter Editor Pro** window. The system prompts, *Recent changes have not been generated. Generate now?* Click **No** to continue with the next flow.

Figure 30. Generated Simulation Design Example File Structure

Note: The External Memory Interfaces Agilex 7 M-Series FPGA IP currently supports only the VCS, ModelSim/QuestaSim, and Xcelium simulators.

Note: If you don't select the **Simulation** or **Synthesis** checkbox, the destination directory contains only Platform Designer design files, which the Quartus Prime software cannot compile directly, but which you can view or edit in the Platform Designer. In this situation you can run the following commands to generate synthesis and simulation file sets:

- To create an Quartus Prime software-compilable project, run the

```
quartus_sh -t make_qii_design.tcl
```

  script in the destination directory.

- To create a simulation project, run the

```
quartus_sh -t make_sim_design.tcl
```

  script in the destination directory.

Note: If you have generated a design example and then make changes to it in the parameter editor, you must regenerate the design example to see your changes implemented. The newly generated design example does not overwrite the existing design example files.

Note: For information on simulating the external memory interface IP, refer to Running Simulation.

2.8. Pin Placement for Agilex 7 M-Series EMIF IP

This topic provides guidelines for pin placement.

Overview

Agilex 7 M-Series FPGAs have the following structure:

- Each device contains up to 8 I/O banks.
- Each I/O bank contains 2 sub-I/O banks.
- Each sub-I/O bank contains 4 lanes.
- Each lane contains 12 general-purpose I/O (GPIO) pins.

General Pin Guidelines

The following are general pin guidelines.

Note: For more detailed pin information, refer to the *Pin and Resource Planning* section in the protocol-specific chapter for your external memory protocol, in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.

- Ensure that the pins for a given external memory interface reside within the same I/O row.
- Interfaces that span multiple banks must meet the following requirements:
  - ☐ The banks must be adjacent to one another. For information on adjacent banks, refer to the *EMIF Architecture: I/O Bank* topic in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.
- All address and command and associated pins must reside within a single sub-bank.
- Address and command and data pins can share a sub-bank under the following conditions:
  - ☐ Address and command and data pins cannot share an I/O lane.
  - ☐ Only an unused I/O lane in the address and command bank can contain data pins.

Table 19. General Pin Constraints

Signal Type	Constraint
Data Strobe	All signals belonging to a DQ group must reside in the same I/O lane.
Data	Related DQ pins must reside in the same I/O lane. For protocols that do not support bidirectional data lines, read signals should be grouped separately from write signals.
Address and Command	Address and Command pins must reside in predefined locations within an I/O sub-bank.

Adjacent Banks

For banks to be considered adjacent, they must reside in the same I/O row. To determine if banks are adjacent, refer to the *EMIF Architecture: I/O Bank* topic in the External Memory Interfaces Agilex 7 M-Series FPGA IP User Guide.

Pin Assignments

To determine locations for all EMIF I/O pins you should refer to the pin table for your device. When referring to the pin table, the bank numbers, I/O bank indices, and pin names are provided.

If you are using the NoC, refer to the Agilex 7 M-Series FPGA Network-on-Chip (NoC) User Guide for information on EMIF-NoC placement.

You can perform pin assignments in a variety of ways. The recommended approach is to manually constrain some interface signals and let the Quartus Prime Fitter handle the rest. This method consists of consulting the pin tables to find legal positions for some of the interface pins and assigning them through the `.qsf` file that is generated with the EMIF design example. For this method of I/O placement, you must constrain the following signals:

- CK0
- PLL reference clock

Based on the above constraints, the Quartus Prime Fitter rotates pins within each lane as necessary.

2.9. Compiling the Agilex 7 M-Series EMIF Design Example

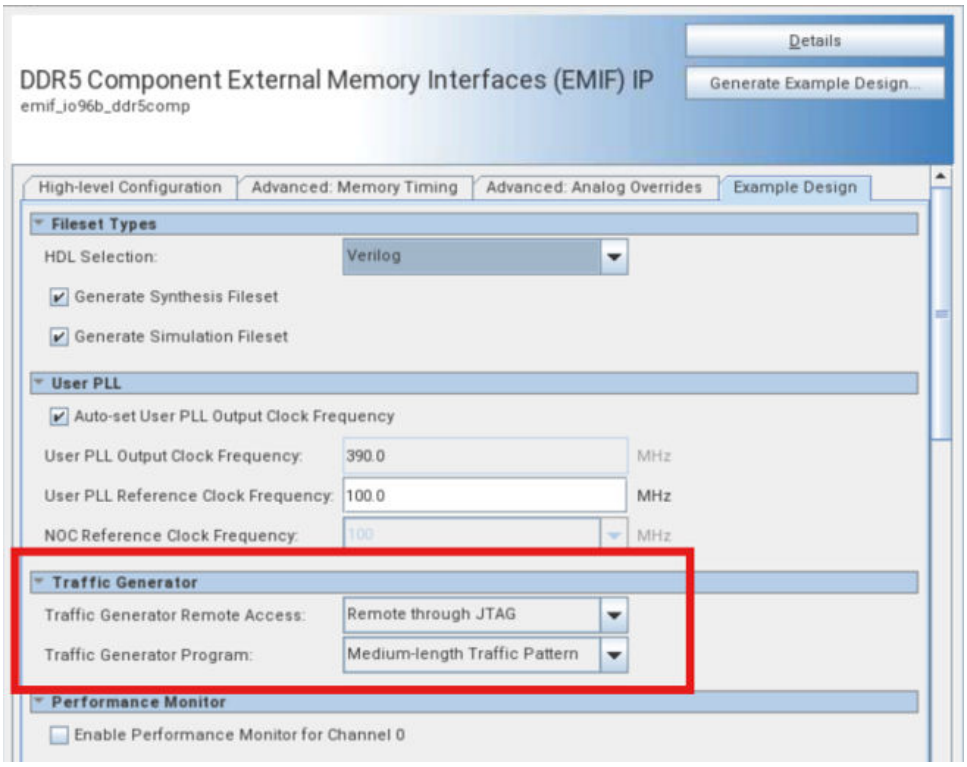
After you have made the necessary pin assignments in the `.qsf` file, you can compile the design example in the Quartus Prime software.

1. Navigate to the Quartus Prime folder containing the design example directory.
2. Open the Quartus Prime project file, (`.qpf`).
3. To begin compilation, click **Processing ➤ Start Compilation**. The successful completion of compilation generates a `.sof` file, which enables the design to run on hardware.

2.10. Using the EMIF Design Example with the Test Engine IP

The test engine IP is a software-programmable AXI traffic generator that generates a configurable pattern of reads and writes to a programmable memory address range. You can modify the length of the default traffic on the **Design Example** tab.

Figure 31. Using the Test Engine IP



Follow these steps to use the EMIF Design Example with the Test Engine IP.

1. Change directory to the location where the `.bin` folder resides. (By default, this is the /qii folder.)
2. At the command line, type the following:

```
system-console --script=<path to testengine_library.tcl> --sof=<path to
sof_file> --update=1 --n-loops=1
```

   Example:

```
system-console --script=hydra_sw/testengine_library.tcl --sof=ed_synth.sof --
update=1 --n-loops=1
```

   This example reprograms the driver with the traffic pattern in the `.bin` folder, and runs the traffic test for 1 loop. If you do not want to reprogram the driver, change `--update=1` to `--update=0`.

The main advantage of the new test engine IP is that you don't need to recompile your entire design when you change or modify the traffic pattern. You need only regenerate the test engine pattern binary files. For additional information on test engine IP functions, refer to the `testengine_library.tcl` file.

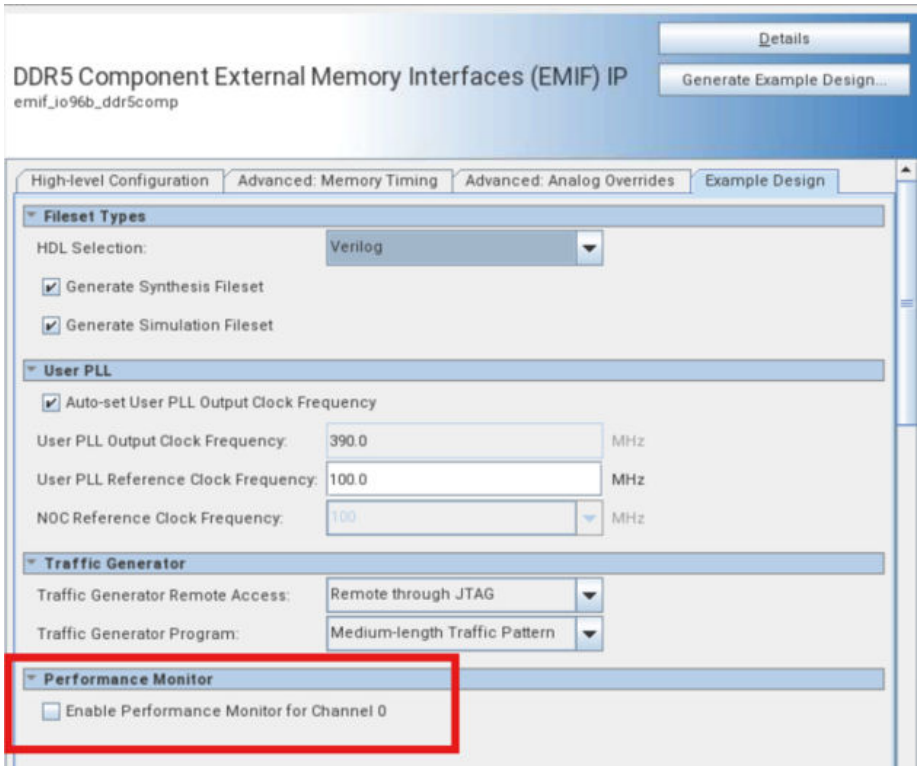
2.11. Generating the EMIF Design Example with the Performance Monitor

You can use the Performance Monitor to check the performance metrics of the EMIF interface.

1. When generating the design example, click the **Enable performance monitor** checkbox. This parameter causes the generated design example to include a performance monitor, known as the *PMON Performance Monitor FPGA IP*, which allows you to measure performance on an AXI4 mainband interface. The performance monitor is a synthesizable block consisting of control and status registers that let you configure and modify the performance metrics of the EMIF Interface. PMON allows you to measure the following performance metrics:
   - Read latency
   - Write latency
   - Read efficiency
   - Write efficiency
   - Overall efficiency
   - Subchannel efficiency
   - Subchannel back pressure
   - Expected transactions in subchannel

   Note: Do not run any traffic on the interface that PMON is monitoring, while you are configuring PMON. Failure to observe this restriction can result in inaccurate measurements.

   Note: The Performance Monitor works only for channel 0.



2. To open the design example, click **File ➤ Open project**, and navigate to the Quartus Prime Pro Edition software folder containing the design example directory, such as:

   ```
   <project_directory>/<example_design_directory>/qii/ed_synth.qpf
   ```

   Click **Open**.

3. You can make the necessary pin assignments in the `.qsf` file or in the Pin Planner.
4. To begin compilation, click **Processing ➤ Start Compilation**. The successful completion of compilation generates a `.sof` file, which you can use to run the design on hardware
5. Load the `pmon_library.tcl` file in the system-console by entering the following command:

   ```
   source pmon_library.tcl
   ```

6. Set the metric configuration on desired AXI4 performance monitors:

   ```
   pmon_set_all basic_eff
   ```

7. Clear the counters and internal state of all performance monitors by typing the following command:

   ```
   pmon_reset_counter_data_all
   ```

8. Run traffic over the interface by typing:

   ```
   testengine_run
   ```

9. Read the efficiency metrics on the AXI4 performance monitor by typing:

```
pmon_read_all
```

3. Design Example Description for External Memory Interfaces Agilex 7 M-Series FPGA IP

When you parameterize and generate your EMIF IP you can specify that the system create directories for simulation and synthesis file sets, and generate the file sets automatically.

If you set **Simulation** or **Synthesis** to **True** on the **Example Design** tab, the system creates a complete simulation file set and/or a complete synthesis file set, in accordance with your selection.

3.1. Synthesis Design Example

The synthesis design example contains the major blocks shown in the figure below.

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

- A traffic generator is a synthesizable AXI4 driver that implements a hard-coded pattern and monitors the data read from the memory to ensure that it matches the written data, and asserts a failure otherwise. You can select different traffic generator programs when generating the example design. Each program performs the following transactions:
  - ☐ Single write and read (with AxLEN=*axlen_a*[1]) [2]
  - ☐ Single write and read (with AxLEN=*axlen_b*[1]) [2]
  - ☐ Sequential address[3] block of *m*[4] writes and *m*[4] reads (with AxLEN=*axlen_a*[1])
  - ☐ Sequential address[3] block of *m*[4] writes and *m*[4] reads (with AxLEN=*axlen_b*[1])
  - ☐ Random address[5] block of *m*[4] writes and *m*[4] reads (with AxLEN=*axlen_a*[1])

1. The axlen_a and axlen_b values depend on the memory technology.
2. Repeated for 3 loops in the traffic generator program = Short.
3. Sequential Address pattern starts at address=0, and increments by (AXI_DATA_WIDTH/8)*(AxLEN+1) on each transaction.
4. The *m* value depends on the traffic generator program.
5. Random address pattern starts at address=0, and uses pseudo-random addresses.

Table 20. axlen_a and axlen_b for Different memory Technologies

Technology	axlen_a	axlen_b	Notes
DDR4	0	0	
DDR5	1	0	axlen_b=0 results in Read-Modify-Write or data-masking on the memory side.
LPDDR4	3	3	
LPDDR5	1	1	

Table 21. *m* value in Different Traffic Generator programs

Traffic Generator Program	*m*
Short	128
Medium	512
Long	4096

- An instance of the memory interface, which includes:
  - ☐ A memory controller which implements all the memory commands and protocol-level requirements.
  - ☐ The PHY, which serves as an interface between the memory controller and external memory devices to perform read and write operations.

Figure 32. Synthesis Design Example Sync Fabric Mode

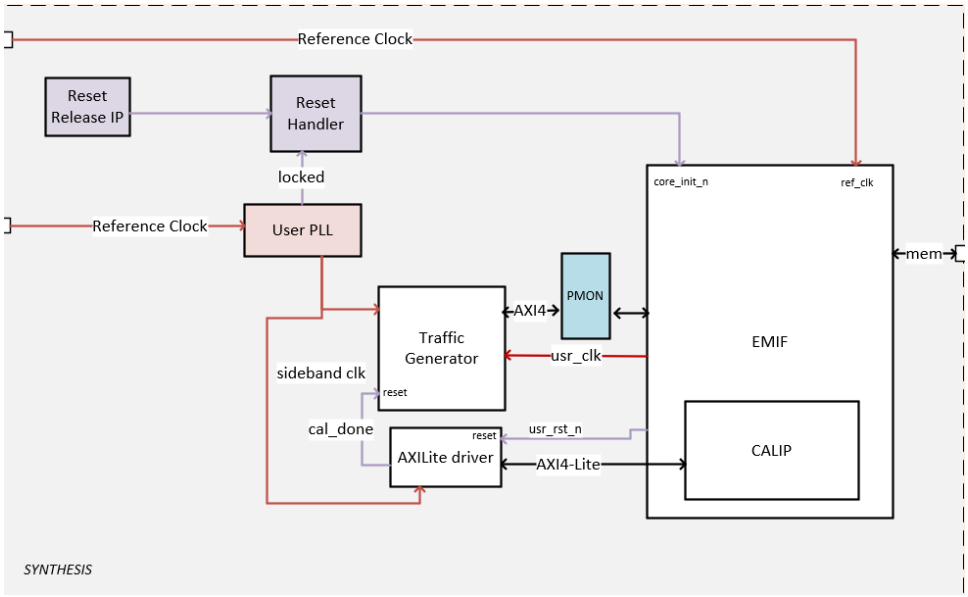


Figure 33. Synthesis Design Example Async Fabric Mode

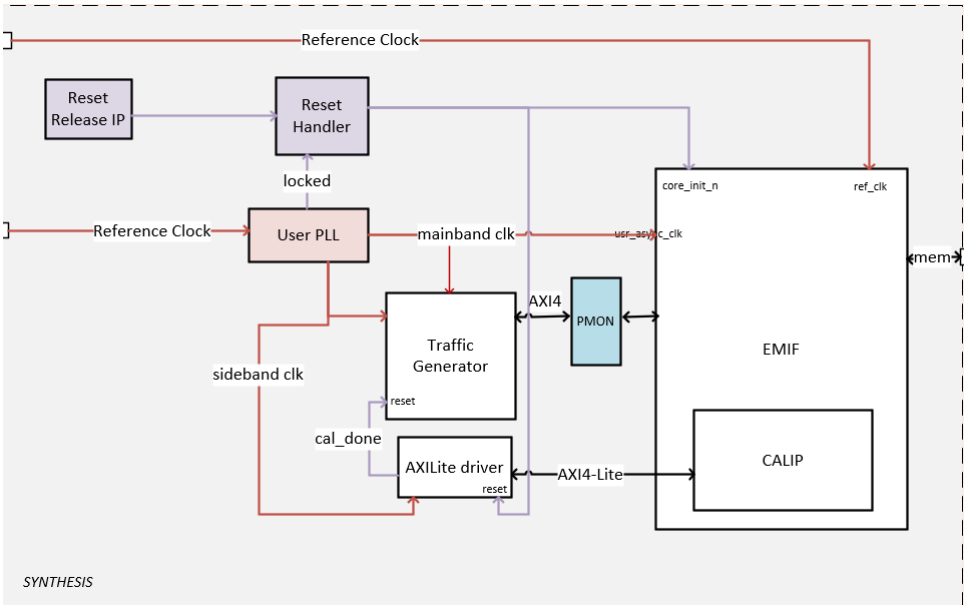
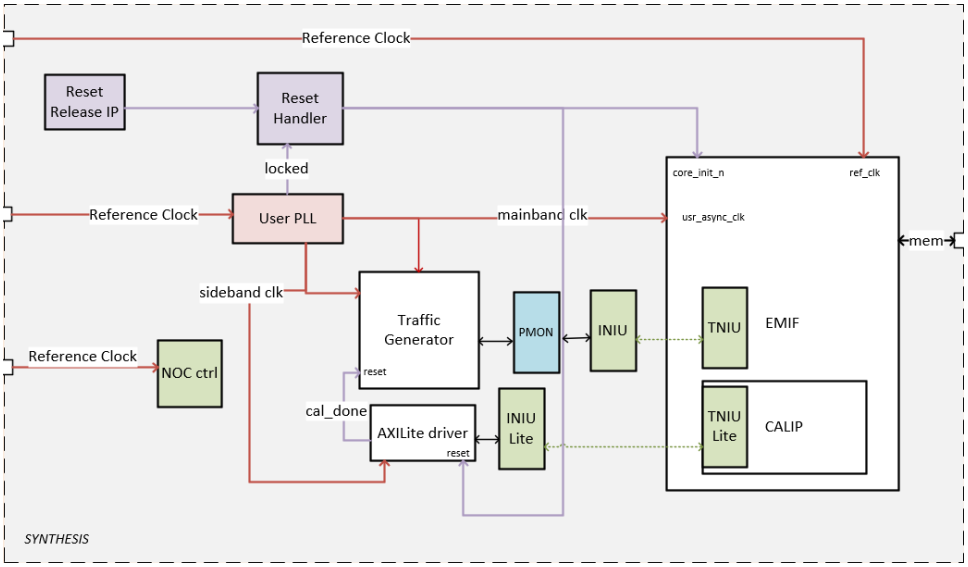


Figure 34. Synthesis Design Example NoC Fabric Mode



3.2. Simulation Design Example

The simulation design example contains the major blocks shown in the following figure.

- An instance of the synthesis design example. As described in the previous section, the synthesis design example contains a traffic generator and an instance of the memory interface.
- A memory model, which acts as a generic model that adheres to the memory protocol specifications. Frequently, memory vendors provide simulation models for their specific memory components that you can download from their websites.

Figure 35. Simulation Design Sync Fabric Mode

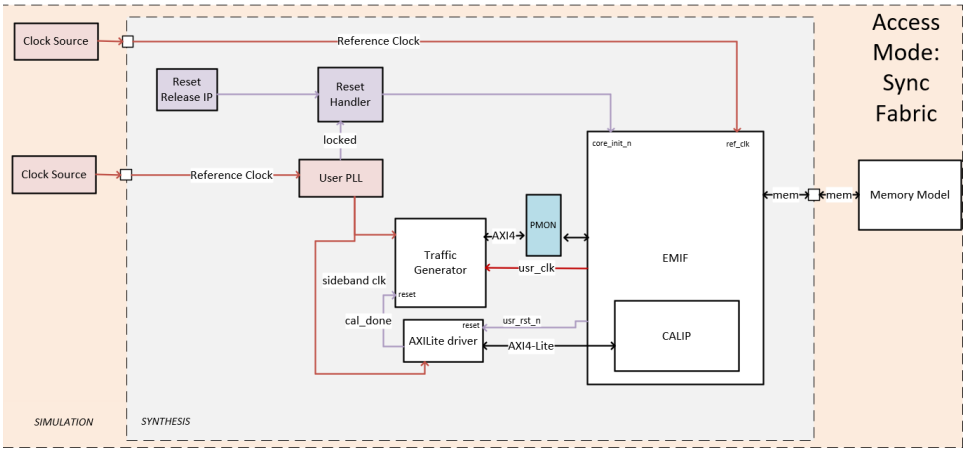


Figure 36. Simulation Design Async Fabric Mode

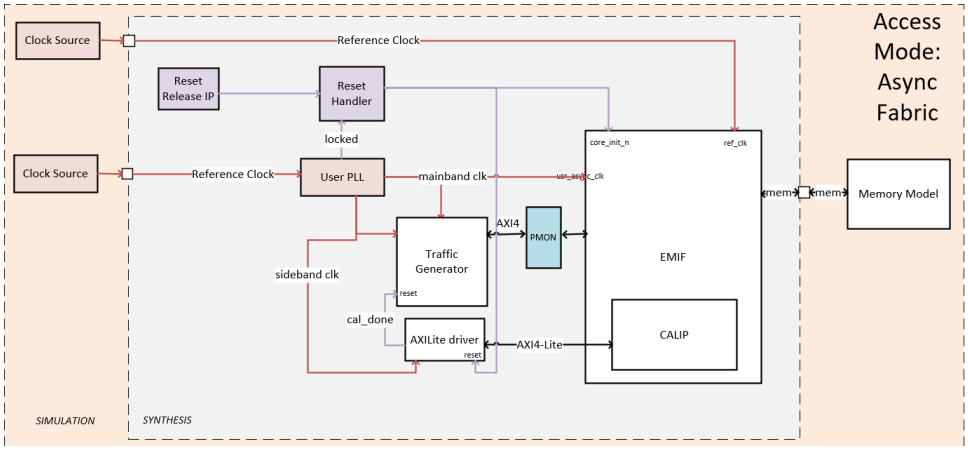
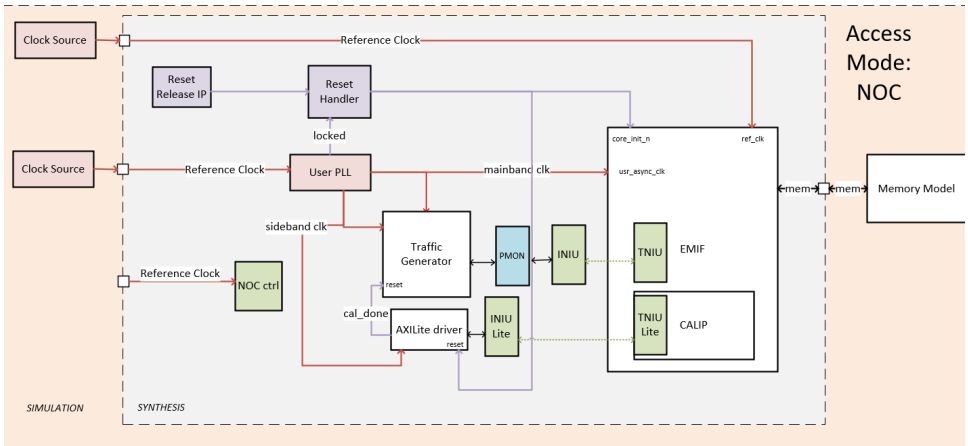


Figure 37. Simulation Design NOC Mode



Note: You must place `ref_clk_usr_pll`. Refer to the *Agilex 7 M-Series FPGA Network-on-Chip (NoC) User Guide* for information on how to add NoC connectivity and address mapping to the simulation netlist.

If a user PLL shares an I/O bank with EMIF, you may additionally need to specify an operating voltage I/O standard. The voltage of the pin depends on the operating voltage of the EMIF occupying that bank, which, in turn, depends on the memory protocol, as follows:

- DDR4 = 1.2V
- DDR5 = 1.1V
- LPDDR5 = 0.7V

For example, for a PLL whose reference clock is in the same bank as a DDR5 EMIF, the assignment show look like this:

```
set_instance_assignment -name IO_STANDARD "1.1V True Differential
Signaling" -to "ref_clk_usr_pll"
```

3.2.1. Running Simulation

To run a simulation, navigate to the simulation directory `<example_design_directory>/sim/ed_sim/` and run the simulation script of your choice.

For ModelSim* SE and Siemens* EDA QuestaSim*- Intel FPGA Edition Simulators

Working directory: `<example_design_directory>/sim/ed_sim/mentor`

1. Invoke vsim by typing `vsim`, which launches a window where you can run the following commands.
2. Change directory to the design example simulator directory (**File ➤ Change directory**.
3. Run the following commands on the transcript terminal:
   a. Do: `source msim_setup.tcl`
   b. `ld_debug`
   c. `run -all`
4. A successful simulation ends with the following message: *Simulation stopped due to successful completion!*

For VCS Simulator

Working directory: `<example_design_directory>/sim/ed_sim/synopsys/vcs`

1. On a single line, type:

```
sh vcs_setup.sh USER_DEFINED_COMPILE_OPTIONS="" USER_DEFINED_ELAB_OPTIONS="-
xlrm\ uniq_prior_final" USER_DEFINED_SIM_OPTIONS=""
```

2. A successful simulation ends with the following message: *Simulation stopped due to successful completion!*

To run a simulation in interactive mode, follow these steps:

Note: If you have already generated a `simv` executable in noninteractive mode, delete the `simv` and `simv.diadir`.

1. Open the `vcs_setup.sh` file and add a debug option to the VCS command: `vcs -debug_access+r`
2. Compile the design example: `sh vcs_setup.sh USER_DEFINED_ELAB_OPTIONS="- xlrm\ uniq_prior_final" SKIP_SIM=1`
3. Start the simulation in interactive mode: `simv -gui &`

Cycle-Accurate NoC Simulation

EMIF IP designs that use the NoC can enable a cycle-accurate NoC simulation model; to do so, follow these steps:

1. Compile your ED project up to the Plan stage of the Fitter.
2. The system creates a file called `noc_sim_defparams.inc` in the `qii/` folder.
3. Modify the `noc_sim_defparams.inc` file, to add the line `` `define SIM_TOP_PATH `` before `initial begin`, as shown:

```
`define SIM_TOP_PATH ed_sim
initial begin
  #1;
  $display("Using NOC accurate model");
```

4. Save the `noc_sim_defparams.inc` file, and then paste a copy into your simulator directory, as follows:
   - For ModelSim* SE and Siemens* EDA QuestaSim*- Intel FPGA Edition Simulators: `<`*`example_design_directory`*`>/sim/ed_sim/mentor`
   - For VCS: `<`*`example_design_directory`*`>/sim/ed_sim/synopsys/vcs`.
5. Open the `<example_design_directory>/sim/ed_sim/sim/ed_sim.v` file in an editor, navigate to the bottom of the file, and comment the line `` `include "ed_sim_noc_sim.inc" `` and add `` `include "noc_sim_defparams.inc" `` immediately before the `endmodule` line, as shown below:

```
   //`include "ed_sim_noc_sim.inc"
   `include "noc_sim_defparams.inc"

endmodule
```

The next steps depend on which simulator you are using, as follows:

For ModelSim* SE and Siemens* EDA QuestaSim*- Intel FPGA Edition simulators:

1. In an editor, open the `<`*`example_design_directory`*`>/sim/ed_sim/mentor/msim_setup.tcl` file.
2. Find the line `set USER_DEFINED_COMPILE_OPTIONS "-O0"` and change it to: `set USER_DEFINED_COMPILE_OPTIONS "+define +PERFORMANCE_ACC_NOC_SIM_MODEL_AGILEX"`
3. Save the file and then follow the usual instructions to launch the simulation.

For VCS simulator:

1. In an editor, open the `<example_design_directory>/sim/ed_sim/synopsys/vcs/vcs_setup.sh` file.
2. Find the line `USER_DEFINED_ELAB_OPTIONS=""` and change it to `USER_DEFINED_ELAB_OPTIONS="+define +PERFORMANCE_ACC_NOC_SIM_MODEL_AGILEX"`.
3. Save the file and then follow the usual instructions to launch the simulation.

3.3. Example Design Tab

The parameter editor includes an **Example Design** tab which allows you to parameterize and generate your design examples.

4. Document Revision History for External Memory Interfaces Agilex 7 M-Series FPGA IP Design Example User Guide

Document Version	Quartus Prime Version	IP Version	Changes
2025.09.29	25.3	4.1.0	• Minor editorial changes throughout. • Updated the figure in the *Parameterizing the External Memory Interface for HPS IP* topic.
2025.07.07	25.1.1	4.0.0	• Updated names of IPs throughout. • Updated screenshots throughout. • In the *Quick Start* chapter: ☐ In the *Parameter Editor Guidelines* topic, modified the sentence immediately preceding the *Command-Address Mirroring* figure. ☐ In the *Using Custom Memory Timing and I/O Configuration* topic, modified step 3 of the procedure. ☐ In the *Parameterizing the External Memory Interface for HPS IP* topic, removed a paragraph immediately preceding the procedure. ☐ Added *Back Annotating Pin Placement and I/O Standard Assignments* topic.
2025.03.31	25.1	3.0.0	• In the *Quick Start* chapter: ☐ Modified the table and text in *Example: Swizzling for a 2Ch x32 + ECC interface*. ☐ Modified example titles in *Example: Byte Swizzling for Lockstep Configuration*.
2025.01.13	24.3.1	2.0.0	• In the *Design Example Description* chapter, added NoC simulation information to the *Running Simulation* topic.
2024.11.18	24.3	1.0.0	• In the *About the External Memory Interfaces IP* chapter, updated the table of IPs and associated version numbers. • In the *Quick Start* chapter, removed topics about custom memory presets. • Updated figures and associated text to reflect GUI changes, throughout.
2024.07.08	24.2	6.2.0	• In the *Quick Start* chapter, added the *Example: Swizzling for a 2Ch x32 + ECC interface* topic. • Implemented minor editorial improvements and branding changes throughout.

continued...

© Altera Corporation. Altera, the Altera logo, the 'a' logo, and other Altera marks are trademarks of Altera Corporation. Altera reserves the right to make changes to any products and services at any time without notice. Altera assumes no responsibility or liability arising out of the application or use of any information, product, or service described herein except as expressly agreed to in writing by Altera. Altera customers are advised to obtain the latest version of device specifications before relying on any published information and before placing orders for products or services.
*Other names and brands may be claimed as the property of others.

Document Version	Quartus Prime Version	IP Version	Changes
2024.04.01	24.1	6.1.0	• In the *Quick Start* chapter: ☐ Updated figures in several topics. ☐ Modified the table in the *Agilex 7 M-Series EMIF Parameter Editor Guidelines* topic. ☐ Added *Using the EMIF Design Example with the Test Engine IP* topic.
2023.12.04	23.4	6.0.0	• In the *Quick Start* chapter: ☐ Added *Command-Address Mirroring* to the *Intel Agilex 7 M-Series EMIF Parameter Editor Guidelines* topic. ☐ Added *Generating the EMIF Design Example with the Performance Monitor* topic.
2023.10.02	23.3	5.0.0	• In the *Quick Start* chapter, modified the *Configuring DQ Pin Swizzling* section. • Updated several figures throughout.
2023.06.26	23.2	4.0.0	In the *Quick Start* chapter, made updates to figures and corrections to syntax examples in the *Configuring DQ Pin Swizzling* topic.
2023.04.03	23.1	3.0.0	Initial release.